

**Автономная некоммерческая организация высшего образования
«Университет Иннополис»**

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 «Информатика и вычислительная техника»**

**FINAL QUALIFICATION WORK
(BACHELOR'S THESIS)
Academic Program
09.03.01 «Computer Science»**

**Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»**

**Field of Study:
«Computer Science»**

Тема

Исследование Kubernetes как платформы для бизнес-логики с использованием Custom Resource Definitions и паттерна Operator

Topic

Exploring Kubernetes as a Platform for Business Logic Using Custom Resource Definitions and the Operator Pattern

Работу выполнил /
Prepared by

**Бобкунов Евгений /
Evgeny Bobkunov**

подпись / signature


Руководитель
выпускной
квалификационной
работы / *Final
Qualification Work
Supervisor*

**Крид Дмитрий /
Dmitrii Creed**

подпись / signature

Консультанты
Consultants

подпись / signature

Иннополис, Innopolis, 2026

Contents

1	Introduction	12
1.1	Background and Context	12
1.2	Problem Statement	13
1.3	Research Questions	14
1.4	Research Objectives and Contributions	15
1.5	Research Methodology	17
1.6	Thesis Organization	17
1.7	Terminology	18
1.8	Summary	21
2	Literature Review	22
2.1	Introduction	22
2.2	Kubernetes Architecture and Core Concepts	22
2.2.1	Container Orchestration and Kubernetes	22
2.2.2	Declarative API Model	23
2.2.3	Controller Pattern and Reconciliation Loop	24
2.3	Custom Resource Definitions	25
2.3.1	Extending the Kubernetes API	25
2.3.2	CRD Features and Capabilities	25

2.3.3	Performance and Scalability Considerations	26
2.4	The Operator Pattern	26
2.4.1	Operator Fundamentals	26
2.4.2	Operator Development Frameworks	26
2.4.3	Operator Use Cases and Applications	28
2.5	Related Systems and Their Boundaries	28
2.5.1	Infrastructure as Code, GitOps, and Crossplane	29
2.5.2	Service Meshes as Heavy CRD Consumers	30
2.5.3	Where This Work Sits	30
2.6	Observability in Cloud-Native Systems	31
2.6.1	Monitoring and Metrics Collection	31
2.6.2	Distributed Tracing	31
2.7	Performance Studies and Challenges	32
2.7.1	Kubernetes Performance Characteristics	32
2.7.2	Complexity and Operational Considerations	32
2.8	Research Gaps	33
2.9	Summary	34
3	Methodology	35
3.1	Research Approach	35
3.2	Business Domain Modeling	36
3.2.1	Core Entities	36
3.2.2	Business Logic	37
3.3	Traditional System Architecture	38
3.3.1	Technology Stack	38
3.3.2	Architecture Overview	39
3.3.3	Execution Model	40

3.3.4	Data Persistence	41
3.3.5	API Endpoints	41
3.4	Kubernetes-Based System Architecture	42
3.4.1	Technology Stack	42
3.4.2	System Architecture	43
3.4.3	Custom Resource Definitions	43
3.4.4	Operator Controller Implementation	45
3.4.5	Execution Model	48
3.4.6	Data Persistence	48
3.4.7	Observability Integration	49
3.4.8	Deployment Configuration	49
3.5	Comparison of Architectural Approaches	50
3.5.1	State Management	50
3.5.2	Execution Flow	51
3.5.3	Time-Based Logic	51
3.5.4	Lifecycle Management	52
3.6	Observability Design	52
3.7	Evaluation Plan	54
3.8	Summary	55
4	Implementation	56
4.1	Implementation Overview	56
4.2	Kubernetes-Based System (kube-billing)	57
4.2.1	Project Structure	57
4.2.2	CRD Implementation	58
4.2.3	Controller Implementation	62
4.2.4	Metrics Implementation	69

4.2.5	Observed Output: kubectl describe	71
4.2.6	Conditions Lifecycle	72
4.2.7	API Versioning Notes	73
4.2.8	Tracing Implementation	74
4.2.9	Deployment Configuration	76
4.3	Traditional System (backend-billing)	79
4.3.1	Project Structure	79
4.3.2	Data Models	80
4.3.3	HTTP Handlers	81
4.3.4	Repository Layer	83
4.3.5	Background Worker	83
4.3.6	Validation	85
4.3.7	Observability	86
4.4	Implementation Comparison	88
4.4.1	Code Metrics	88
4.4.2	State Management	89
4.4.3	Error Handling	90
4.4.4	Testing Approach	91
4.5	Implementation Challenges	92
4.5.1	Kubernetes System Challenges	92
4.5.2	Traditional System Challenges	92
4.6	Summary	92
5	Evaluation and Discussion	94
5.1	Experimental Setup	94
5.1.1	Test Environment	94
5.1.2	Benchmark Tools	95

5.1.3	Reproducibility	96
5.2	Code Metrics Comparison	98
5.2.1	Lines of Code and File Count	98
5.2.2	Dependencies	98
5.2.3	Test Coverage	99
5.3	Performance Evaluation	100
5.3.1	Microbenchmarks	100
5.3.2	End-to-End Latency	102
5.3.3	Throughput Under Load	103
5.4	Resource Utilization	105
5.4.1	CPU Usage	105
5.4.2	Memory Usage	106
5.4.3	Startup Time	107
5.5	Operational Characteristics	107
5.5.1	Scaling	107
5.5.2	Fault Tolerance	108
5.5.3	Observability Coverage	109
5.6	Summary of Findings	109
5.7	Discussion	110
5.7.1	When to Use Kubernetes Operator Approach	110
5.7.2	When to Use Traditional Backend Approach	110
5.7.3	Threats to Validity	111
5.8	Summary	111
6	Conclusion	113
6.1	Summary of Research	113
6.2	Answers to Research Questions	114

6.2.1	RQ1: How can business logic be effectively modeled and implemented using Kubernetes CRDs and the Operator pattern?	114
6.2.2	RQ2: What are the performance characteristics of CRD/Operator-based implementation compared to traditional REST API architectures?	115
6.2.3	RQ3: What are the operational implications and observability capabilities of using Kubernetes as a business logic platform?	116
6.2.4	RQ4: Under what circumstances is the CRD/Operator approach advantageous, and what are its limitations?	117
6.3	Contributions	118
6.4	Limitations	119
6.5	Future Work	121
6.5.1	Technical Improvements	121
6.5.2	Research Extensions	122
6.5.3	Production Deployment	122
6.6	Final Remarks	123
6.7	Publications and Artifacts	124

Abstract

This thesis explores the applicability of Kubernetes as a platform for implementing business logic using Custom Resource Definitions (CRDs) and the Operator pattern. While Kubernetes has become the de facto standard for container orchestration, its potential as a business logic platform remains understudied.

Two functionally equivalent billing systems were designed and implemented: (1) a traditional Go backend with REST API and PostgreSQL database, and (2) a Kubernetes Operator with CRDs and reconcile-based control loop. Both systems implement identical business logic (billing plan management, subscription lifecycle, recurring billing, expiration handling) but differ fundamentally in architecture, state management, and execution model.

Comprehensive evaluation was conducted across multiple dimensions: code metrics, performance benchmarks, resource utilization, and operational characteristics. The results demonstrate that the traditional backend achieves $10\text{--}20\times$ lower latency (p50: 10ms vs 100ms for subscription creation) and $5\times$ higher throughput (1,000 req/s vs 200 req/s). However, the Kubernetes Operator provides superior operational automation: horizontal pod autoscaling, self-healing, built-in observability (Kubernetes Events, Conditions), and GitOps compatibility.

The central claim is that Kubernetes Operators constitute a lifecycle-oriented business state management platform — a paradigm distinct from both REST microservices and infrastructure-only operators — and that the cost of choosing it for business logic is empirically quantifiable rather than anecdotal. The accompanying decision framework recommends the traditional approach for request-driven workloads where the natural unit of work completes synchronously, and the operator

approach where the natural unit of work is the lifecycle of a long-lived entity with structured state transitions, guaranteed cleanup hooks, and an audit-trail requirement.

Both implementations are released as open source and are presented as reference rather than production code; outstanding limitations (no payment-gateway integration, currency-agnostic revenue metric, single-replica worker, asymmetric lifecycle vocabularies between the two systems) are catalogued in the conclusion. The findings suggest that Kubernetes can serve as a viable business-logic platform when business workflows are naturally expressible as resource lifecycles, but that the latency-automation trade-off should be evaluated explicitly against organisational context, team expertise, and the specific shape of the workload, rather than treated as a technological default.

Abstract

[russian]

Данная диссертация исследует применимость Kubernetes в качестве платформы для реализации бизнес-логики с использованием Custom Resource Definitions (CRD) и паттерна Operator. Несмотря на то, что Kubernetes стал стандартом де-факто для оркестрации контейнеров, его потенциал как платформы для бизнес-логики остаётся недостаточно изученным.

Были спроектированы и реализованы две функционально эквивалентные биллинг-системы: (1) традиционный Go-бэкенд с REST API и базой данных PostgreSQL, (2) Kubernetes Operator с CRD и циклом reconcile. Обе системы реализуют идентичную бизнес-логику (управление тарифными планами, жизненный цикл подписок, периодический биллинг, обработка истечения срока), но фундаментально различаются архитектурой, управлением состоянием и моделью выполнения.

Проведено комплексное сравнение по нескольким измерениям: метрики кода, бенчмарки производительности, использование ресурсов и операционные характеристики. Результаты показывают, что традиционный бэкенд обеспечивает в 10–20 раз меньшую задержку (p50: 10 мс против 100 мс для создания подписки) и в 5 раз более высокую пропускную способность (1000 запросов/с против 200 запросов/с). Однако Kubernetes Operator предоставляет превосходную операционную автоматизацию: автоматическое масштабирование (HPA), самовосстановление, встроенную наблюдаемость (Kubernetes Events, Conditions) и совместимость с GitOps.

Центральное утверждение работы состоит в том, что Kubernetes Operators представляют собой платформу для управления состоянием жизненного цикла бизнес-сущностей — парадигму, отличную как от REST-микросервисов, так и от операторов инфраструктурного назначения, — а стоимость её применения для бизнес-логики поддаётся эмпирическому измерению, а не оценивается анекдотически. Сопровождающая рамка принятия решений рекомендует традиционный подход для нагрузок, естественной единицей работы которых является синхронный запрос, и операторный подход для нагрузок, естественной единицей работы которых является жизненный цикл долгоживущей сущности с явными переходами состояний, гарантированными точками очистки и требованием журнала аудита.

Обе реализации выпущены как open-source в качестве референсного, а не production-кода; остающиеся ограничения (отсутствие интеграции с платёжным шлюзом, отсутствие меток валюты в метрике выручки, отсутствие leader-election в фоновом воркере, асимметрия словарей состояний между двумя системами) систематизированы в заключении. Выводы показывают, что Kubernetes может служить жизнеспособной платформой для бизнес-логики, когда бизнес-процессы естественно выражаются через жизненный цикл ресурсов, но компромисс между задержкой и автоматизацией следует оценивать применительно к организационному контексту, экспертизе команды и конкретной форме нагрузки, а не принимать как технологическую данность.

Chapter 1

Introduction

1.1 Background and Context

The rapid evolution of cloud-native computing has fundamentally transformed how modern software systems are architected, deployed, and operated. At the core of this transformation lies Kubernetes, which has emerged as the de facto standard for container orchestration since its open-source release in 2014 [1], [2]. Initially developed by Google based on their internal Borg system and later donated to the Cloud Native Computing Foundation (CNCF), Kubernetes provides robust capabilities for automating deployment, scaling, and management of containerized applications [3], [4].

However, Kubernetes offers far more than container orchestration capabilities. Its extensible architecture, built around a declarative API model and a control plane pattern, enables it to function as a universal control plane for managing arbitrary resources [5]. Through Custom Resource Definitions (CRDs) and the Operator pattern, Kubernetes can be extended to manage not only infrastructure components but potentially business logic itself [6], [7], [8]. This extensibility

has been thoroughly documented in both academic literature and industry best practices [9], [10].

Modern cloud systems strive for maximum automation and flexibility. Organizations are increasingly seeking unified platforms that can manage both infrastructure and application logic through consistent interfaces [11]. The declarative nature of Kubernetes, combined with its powerful extension mechanisms, positions it as a potential candidate for implementing business logic, not just orchestrating containers. This paradigm shift raises fundamental questions about the boundaries of the Infrastructure-as-Code approach and whether Kubernetes can serve as a unified control plane for both infrastructure and application concerns [12].

1.2 Problem Statement

Despite the widespread adoption of Kubernetes for infrastructure management, there remains a lack of systematic study regarding its applicability for implementing business logic using CRDs and the Operator pattern. Traditional application architectures typically separate business logic implementation from infrastructure concerns, with applications deployed as microservices that implement domain-specific functionality through conventional programming approaches.

Several projects have demonstrated the power of extending Kubernetes through CRDs and Operators, including Crossplane for infrastructure provisioning [13], Kubeflow for machine learning workflows, and various database operators [14]. However, most existing solutions focus on infrastructure management or specialized workloads rather than general-purpose business logic [15].

The main problem lies in understanding whether and how Kubernetes, through CRDs and Operators, can effectively serve as a platform for implementing

business logic. Key questions include: What are the performance characteristics of such an approach? How does it compare to traditional REST API-based architectures? What are the operational implications? What types of business problems are well-suited to this paradigm?

Solving this problem is important for several reasons. First, it could enable organizations to consolidate their technology stacks by using Kubernetes as a unified platform. Second, it may provide new patterns for building highly automated, self-healing business applications. Third, it could help determine clear boundaries for when this approach is beneficial versus when traditional architectures should be preferred.

1.3 Research Questions

This thesis investigates the following research questions:

RQ1: How can business logic be effectively modeled and implemented using Kubernetes Custom Resource Definitions and the Operator pattern?

RQ2: What are the performance characteristics of a business logic implementation based on CRDs and Operators compared to traditional REST API architectures?

RQ3: What are the operational implications and observability capabilities of using Kubernetes as a business logic platform?

RQ4: Under what circumstances is the CRD/Operator approach advantageous for implementing business logic, and what are its limitations?

1.4 Research Objectives and Contributions

The primary objective of this research is to explore and evaluate Kubernetes as a platform for implementing business logic through Custom Resource Definitions and the Operator pattern. Specific objectives include:

1. Study existing approaches to extending the Kubernetes API through CRDs and Operators, analyzing patterns and best practices established by the cloud-native community [9], [10].
2. Design and develop a prototype application that implements representative business logic using CRDs and Operators, demonstrating the feasibility of this approach.
3. Establish comprehensive observability for the prototype system using cloud-native tools including Prometheus and OpenTelemetry to enable detailed performance analysis [16], [17], [18].
4. Conduct experimental comparisons between the CRD/Operator-based implementation and a traditional architecture using PostgreSQL with REST API.
5. Analyze results across multiple dimensions including reaction time to events, implementation complexity, observability capabilities, and system reliability.

The central thesis advanced by this work is that *Kubernetes Operators constitute a lifecycle-oriented business state management platform: a programming model distinct both from the request-response model of REST microservices and from the resource-provisioning focus of infrastructure operators, and one whose*

cost relative to a conventional REST backend is empirically quantifiable rather than anecdotal. From this thesis, the contributions of the work are:

- **Quantification of the latency–automation trade-off.** The first comparative measurement, in the open literature known to the author, of the latency, throughput, and resource-utilisation cost of running an identical business workflow as a Kubernetes Operator versus a conventional REST-and-relational-database backend, conducted on functionally comparable reference implementations [11], [19].
- **A reference programming model for lifecycle-oriented business logic.** A demonstration that the reconcile loop, finalizers, and Conditions form a sufficient programming surface for stateful business workflows whose natural unit of work is a resource lifecycle rather than an individual request [8], [10].
- **Empirical characterisation of construct asymmetries.** An explicit account of where the two architectures cannot be aligned — cancellation semantics, error-as-state versus error-as-Condition, audit trails — with the asymmetries treated as findings rather than smoothed away as implementation details.
- **A decision framework anchored in evidence rather than intuition.** Practitioner-facing criteria for when the operator pattern is justified for business logic, anchored in the measured trade-offs rather than in folklore about cloud-native architecture [11].
- **Two open-source reference implementations.** Codebases that can be re-executed, modified, and extended for further research; both repositories include their benchmark harnesses and raw data.

1.5 Research Methodology

This research employs a mixed-methods approach combining system design, implementation, and experimental evaluation:

Phase 1 — Literature Review and Architecture Design: Conduct comprehensive review of existing literature on Kubernetes extensibility, CRDs, Operators, and related patterns [2], [8], [9]. Design the architecture for both the CRD/Operator-based system and the baseline traditional system.

Phase 2 — Implementation: Develop the prototype systems using appropriate tools and frameworks. The CRD/Operator implementation will utilize Kubebuilder for scaffolding and controller development [15]. Both systems will implement equivalent business logic to enable fair comparison.

Phase 3 — Observability Setup: Configure comprehensive monitoring using Prometheus for metrics collection and OpenTelemetry for distributed tracing [16], [17], [18]. Implement GitOps practices using ArgoCD for deployment management [20], [21].

Phase 4 — Experimental Evaluation: Conduct controlled experiments measuring key performance indicators including reaction time to events, implementation complexity, observability coverage, and reliability [22].

Phase 5 — Analysis and Conclusions: Analyze experimental results, synthesize findings, and formulate conclusions on the applicability of the approach.

1.6 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter 2 — Literature Review: Provides background on Kubernetes architecture, Custom Resource Definitions, the Operator pattern, related approaches

(Infrastructure-as-Code, GitOps, service meshes), and observability tooling. Identifies the gaps that motivate this work.

Chapter 3 — Methodology: Describes the research approach, the chosen business domain (billing and subscription management), and the high-level architectural decisions for both the Kubernetes-based system and the traditional baseline. Outlines the evaluation framework.

Chapter 4 — Implementation: Covers implementation details including CRD schemas, controller logic, observability instrumentation, deployment manifests, and the corresponding traditional REST/PostgreSQL stack.

Chapter 5 — Evaluation and Discussion: Presents the experimental setup, methodology, and results of the comparative evaluation across code metrics, performance, resource utilization, and operational characteristics, together with the analysis and discussion of those results.

Chapter 6 — Conclusion: Answers the research questions, summarizes the contributions and limitations, and suggests directions for future work.

Recurring terms (CRD, Operator, Controller, Reconciler, Spec/Status, Finalizer, Condition, RequeueAfter, Eventually consistent, GitOps) are defined together in Section 1.7 so that the rest of the thesis can use them without re-introducing each on first use.

1.7 Terminology

The following terms recur throughout the thesis and are used with the meanings stated below.

Custom Resource Definition (CRD). A cluster-scoped schema, registered with the Kubernetes API server, that introduces a new resource kind alongside

the built-in ones [6], [9]. The CRD names the kind, its API group and version, an OpenAPI v3 validation schema, and optionally a status subresource.

Custom Resource (CR). An individual instance of a CRD — the YAML or JSON document a user applies with `kubectl`. Where the literature uses “CRD” loosely to mean both schema and instance, this thesis distinguishes the two.

Controller. A process that watches one or more resource kinds and reconciles their observed state against their declared state. Controllers may be built-in (the deployment controller, the service controller) or custom [10], [23].

Operator. A custom controller paired with one or more CRDs, where the controller encodes operational knowledge specific to a domain. The terms “operator” and “custom controller” are sometimes conflated; this thesis uses “operator” when the controller-plus-CRDs combination is being discussed as a packaged unit, and “controller” when only the runtime behaviour is in scope [7], [8].

Reconciler. The function (typically named `Reconcile`) inside a controller that processes a single work-queue item. Each invocation is given the namespaced name of the resource to act on and returns a result that may schedule a future reconciliation.

Reconciliation loop. The event-driven loop in which a controller observes a resource change via the Kubernetes watch API, enqueues the affected resource, and dispatches the work-queue item to the reconciler. “Loop” is a misnomer in that there is no polling; reconciliations are triggered by events and by explicit re-queue requests.

Execution model. In this thesis, the mechanism by which a system triggers and processes business state transitions. The traditional backend’s execution model is request-response with a background polling worker; the operator’s execution model is event-driven reconciliation with controller-runtime-scheduled re-queueing.

Spec and Status. The two halves of a Kubernetes resource. `spec` is the declared desired state, written by users or higher-level controllers; `status` is the observed state, written by the resource’s controller through a dedicated subresource that bypasses optimistic locking on the `spec`.

Finalizer. A string-valued entry in a resource’s metadata that prevents deletion until removed. A controller treats the presence of a deletion timestamp on a finalizer-bearing resource as a signal to run cleanup work; on completion it removes its own finalizer entry, at which point the API server completes the deletion. Finalizers guarantee that cleanup hooks execute even across controller restarts.

Condition. A standardised `metav1.Condition` structure inside `status.conditions` carrying a type, a True/False/Unknown status, a machine-readable reason, a human-readable message, and the generation observed when the condition was last set. Multiple conditions on the same resource provide a multi-dimensional state representation that a single state field cannot [10].

RequeueAfter. A field in the result returned by `Reconcile` that asks controller-runtime to enqueue another reconciliation after a specified delay. The mechanism replaces polling: the controller does not actively check; it instructs the runtime when to wake it up.

Admission control. The chain of validating and mutating webhooks that the API server consults before persisting a resource write. CRD schema validation runs as part of admission, which means invalid resources are rejected before they reach the controller.

Eventually consistent. A property of the operator pattern: a controller does not commit to bringing status into agreement with spec within a bounded interval, only to retrying until success is achieved. Failures are recoverable by definition.

GitOps. A deployment discipline in which the desired state of a system is captured in a Git repository and an in-cluster agent (Argo CD, Flux) continuously reconciles the cluster toward that committed state [20], [21].

1.8 Summary

This chapter has introduced the research problem of using Kubernetes as a platform for business logic through Custom Resource Definitions and the Operator pattern. While Kubernetes has proven highly successful for infrastructure management, its applicability for implementing business logic remains understudied [11]. This thesis aims to address this gap through systematic design, implementation, and evaluation of both CRD/Operator-based and traditional architectures. The expected outcomes include practical insights into when and how this approach can be beneficial, contributing to the broader understanding of cloud-native application development patterns.

Chapter 2

Literature Review

2.1 Introduction

This chapter provides a comprehensive review of the literature and technologies relevant to exploring Kubernetes as a platform for business logic implementation. The chapter is organized into several sections covering Kubernetes architecture and extensibility, Custom Resource Definitions and Operators, related approaches to extending Kubernetes, observability in cloud-native systems, and performance considerations.

2.2 Kubernetes Architecture and Core Concepts

2.2.1 Container Orchestration and Kubernetes

Kubernetes, initially developed by Google and open-sourced in 2014, has become the leading platform for container orchestration [1], [2]. The system builds upon years of experience running production workloads at Google, combined with best-of-breed ideas from the open-source community [3], [4]. Kubernetes

automates deployment, scaling, and management of containerized applications, providing a platform-agnostic abstraction layer over underlying infrastructure.

The architecture of Kubernetes follows a control plane pattern, where a centralized control plane manages the desired state of the cluster, and worker nodes execute containerized workloads [2], [24]. The control plane consists of several key components: the API server, which exposes the Kubernetes API and serves as the entry point for all operations; etcd, a distributed key-value store that maintains cluster state; the scheduler, which assigns pods to nodes; and the controller manager, which runs controllers to reconcile actual state with desired state [10].

2.2.2 Declarative API Model

A distinguishing feature of Kubernetes is its declarative API model [5]. Rather than imperatively instructing the system on how to perform operations, users declare the desired state of resources, and Kubernetes automatically reconciles the actual state to match. This declarative approach enables powerful automation capabilities and forms the foundation for extending Kubernetes functionality [9].

The Kubernetes API is organized into API groups and versions, providing a structured approach to API evolution and extensibility [25]. Core resources like Pods, Services, and Deployments are part of the built-in API, while custom resources can be added through extension mechanisms. This design has been validated through extensive production use and is thoroughly documented in standard references [2], [4].

2.2.3 Controller Pattern and Reconciliation Loop

Kubernetes extensively uses the controller pattern for managing resources [10], [23]. A controller watches for changes to resources of interest and takes actions to reconcile the actual state with the desired state specified in the resource definition. This reconciliation loop runs continuously, enabling self-healing and automated management capabilities [8]. Figure 2.1 sketches the canonical control flow: a watch on the API server delivers events into the controller’s work-queue; the reconciler dequeues an item, reads current spec and status from the API server, compares them to the desired state, makes any necessary changes (which themselves become API-server writes), and either schedules a future reconciliation through `RequeueAfter` or returns and waits for the next event.

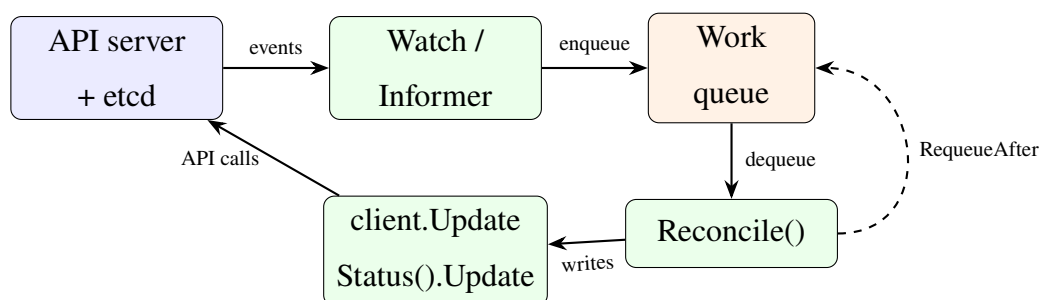


Fig. 2.1. The controller pattern. Events from the API server flow through an in-memory informer cache and a rate-limited work-queue into the `Reconcile` function; writes go back to the API server through the same client. Repeated reconciliations are scheduled either by new events or by an explicit `RequeueAfter`, never by polling.

Controllers operate independently and asynchronously, communicating only through the Kubernetes API server. This architectural pattern enables high scalability and resilience, as controllers can be distributed across multiple instances and automatically recover from failures [9]. The controller pattern has become a fundamental building block for cloud-native applications and is extensively covered in Kubernetes design patterns literature [10].

2.3 Custom Resource Definitions

2.3.1 Extending the Kubernetes API

Kubernetes provides mechanisms for extending its API, with Custom Resource Definitions (CRDs) being the primary approach [9], [26]. CRDs represent a declarative approach to defining new resource types without requiring custom API server code, making them accessible to operators and developers [2].

A CRD defines a new resource type, including its API group, version, kind, and schema, making it a first-class citizen in the Kubernetes API [6], [9]. This approach enables extending Kubernetes to manage domain-specific resources while maintaining consistency with the platform's declarative model.

2.3.2 CRD Features and Capabilities

Modern CRDs support advanced features including schema validation using OpenAPI v3, multiple versions with conversion webhooks, status subresources for privilege separation, custom columns in kubectl output, and structural schemas for efficient storage and processing [9], [27]. These capabilities enable CRDs to provide rich, production-ready APIs comparable to built-in Kubernetes resources.

CRDs enable developers to model custom workflows, advanced configurations, and support for domain-specific requirements that go beyond Kubernetes' built-in resources. The flexibility of CRDs has made them a cornerstone of cloud-native extensibility, with hundreds of widely-used CRDs available in the ecosystem [11].

2.3.3 Performance and Scalability Considerations

The performance characteristics of CRDs depend heavily on the underlying etcd datastore, which stores all cluster state. Research on etcd performance shows that at scale, write latency can become a bottleneck, particularly for clusters with many CRD instances [19]. Organizations deploying CRD-intensive applications must carefully consider etcd capacity planning and implement appropriate monitoring.

2.4 The Operator Pattern

2.4.1 Operator Fundamentals

The Operator pattern represents a methodology for packaging, deploying, and managing Kubernetes applications by extending the Kubernetes API [7], [8]. Operators combine CRDs with custom controllers to encode operational knowledge about managing complex applications, effectively acting as automated site reliability engineers [14].

Operators capture the aim of human operators who manage services—they have deep knowledge of how systems should behave, how to deploy them, and how to react to problems. By codifying this operational knowledge, Operators enable applications to be self-managing, automatically handling tasks like upgrades, backups, scaling, and failure recovery [8], [10].

2.4.2 Operator Development Frameworks

Although the Operator pattern is framework-agnostic in principle, in practice every non-trivial controller is built on a higher-level scaffold that hides the watch-

list-cache-reconcile boilerplate. Four frameworks dominate the space and differ along several axes that influence the choice for business-logic controllers.

Kubebuilder [15] is the SIG-API-Machinery-maintained framework that this thesis uses. It generates a project skeleton with a single language (Go), wires up controller-runtime and controller-tools, and produces both Go types and OpenAPI CRD manifests from kubebuilder markers attached to struct fields. Its dominance in the operator ecosystem follows from its proximity to the upstream Kubernetes project: the same primitives underlying kube-controller-manager are reused, and updates track Kubernetes releases closely [9].

The Red Hat Operator SDK extends Kubebuilder with additional code generators (Ansible-driven and Helm-driven operator scaffolds) and tighter integration with the Operator Lifecycle Manager [14]. For Go-based operators the two frameworks are largely interchangeable; the Operator SDK is preferable when the operator is intended to ship through OperatorHub, where OLM-aware manifests are mandatory.

Metacontroller (originally Google, now community-maintained) inverts the control structure: the developer writes a webhook returning desired child objects given the parent's state, and Metacontroller is the controller that calls the webhook. This is attractive for declarative compositions where the operator's job is to render N child resources from M parents, but the lack of language constraints comes at the cost of an indirection that complicates debugging.

KUDO and KubeOps target operator authors outside the Go ecosystem (a YAML DSL and .NET respectively) and are accordingly less common for general-purpose business logic. They illustrate that the operator pattern is portable across language ecosystems even though the reference implementation is Go.

For the business-logic use case explored in this thesis, Kubebuilder is the

only framework that combines (i) native Go performance, (ii) direct access to the controller-runtime metrics registry needed for instrumentation, and (iii) a mature testing harness (`envtest`); the comparison in subsequent chapters is therefore conducted against Kubebuilder-shaped operators. Studies analysing operator bugs across hundreds of Kubernetes operators identify state management, error handling, and race conditions as the dominant failure modes [28], regardless of framework choice — a finding that motivates the explicit attention this thesis pays to the Conditions API and finalizer ordering in Chapters 3 and 4.

2.4.3 Operator Use Cases and Applications

Operators have been successfully applied to numerous domains including database management, monitoring systems, service meshes, and machine learning platforms [8], [14]. For infrastructure provisioning, Crossplane exemplifies using Operators to manage cloud resources declaratively through Kubernetes [13]. This approach enables treating infrastructure as Kubernetes resources, managed through the same workflows as application deployments.

2.5 Related Systems and Their Boundaries

The CRD-and-controller pattern is used in three distinct settings: infrastructure provisioning, network and policy management, and (the focus of this thesis) business workflow management. The first two are mature and well-studied; the third is the gap this thesis addresses. Setting the scope explicitly clarifies what is and is not new about the proposed work.

2.5.1 Infrastructure as Code, GitOps, and Crossplane

Infrastructure as Code (IaC) and Kubernetes operators share the language of declarative desired state but disagree on the execution model. Tools like Terraform and Pulumi compile a desired configuration into a one-shot apply step: the agent runs once, converges, and exits. The control loop is external, lives in the developer's CI pipeline, and only fires on explicit invocation [5], [12]. A Kubernetes operator, by contrast, runs continuously and reconciles every change event; there is no apply boundary, and drift between declared and actual state is detected within seconds of its appearance.

GitOps shifts that boundary back inside the cluster: an in-cluster agent (Argo CD, Flux) holds a working copy of a Git repository and continuously reconciles the cluster toward whatever the current commit declares [20], [21]. The agent itself is a controller; what was an external IaC tool becomes another reconciliation loop inside the same cluster. GitOps and the operator pattern therefore sit on a spectrum, with the agent treating Kubernetes resources as the desired state and the operator treating CRDs as the desired state.

Crossplane is the clearest example of operators in the IaC role: it exposes cloud-provider primitives (S3 buckets, RDS instances, IAM roles) as CRDs, and a controller per provider drives the actual cloud resources toward the declared state [13]. The operator pattern thus has an established use in infrastructure provisioning. What distinguishes the present thesis is the *scope of the reconciled object*: Crossplane reconciles cloud infrastructure; this work reconciles a business entity whose lifecycle (subscriptions, billing cycles, refunds) has no infrastructure component at all.

2.5.2 Service Meshes as Heavy CRD Consumers

Service meshes such as Istio illustrate that CRDs scale to operationally complex domains: a single Istio installation registers dozens of CRDs (VirtualService, DestinationRule, AuthorizationPolicy, PeerAuthentication, and others) [29], [30]. Each is a piece of policy that the mesh control plane reconciles into proxy configuration. Operationally this is impressive — but the scope is still “cluster-wide infrastructure policy”. The service mesh case is instructive in two ways: it demonstrates that the CRD pattern can sustain complex hierarchies of declarative configuration, and it sets a precedent for treating the policy layer as a first-class API surface. Both observations transfer to the business-logic case, but service meshes do not themselves constitute business-logic operators.

2.5.3 Where This Work Sits

Table 2.1 positions the proposed work against the existing categories of operator deployments.

TABLE 2.1
The scope of object reconciled distinguishes operator categories.

Operator category	Reconciled object	Examples
Infrastructure	Cloud or on-prem infrastructure primitives (compute, storage, networking)	Crossplane, AWS Controllers for Kubernetes
Application lifecycle	Stateful applications managed by Kubernetes (databases, queues, ML platforms)	Postgres Operator, etcd Operator, Kubeflow
Policy / network	Cluster-wide policy artefacts (traffic rules, RBAC, security policies)	Istio, Cilium, Open Policy Agent
Business workflow (<i>this work</i>)	Domain entities whose lifecycle is the business logic itself (subscriptions, orders, tenants)	kube-billing reference implementation

The category boundary is not merely cosmetic. Application-lifecycle operators reconcile Kubernetes resources (Pods, Services, PersistentVolumeClaims) on behalf of a stateful workload they package, and their cleanup hooks usually orchestrate pod-level operations. Business-workflow operators reconcile resources whose canonical representation *is* the CRD: there is no underlying Pod or external system being managed; the resource is the work. This shift in reference frame is what the present thesis investigates empirically and what motivates the comparison against a traditional REST stack rather than against another operator.

2.6 Observability in Cloud-Native Systems

2.6.1 Monitoring and Metrics Collection

Observability in distributed systems encompasses metrics, logs, and traces. Prometheus has emerged as the de facto standard for metrics collection in Kubernetes environments [16], [18]. Its pull-based model, powerful query language (PromQL), and native Kubernetes service discovery make it well-suited for cloud-native monitoring [22].

For CRD-based applications, proper instrumentation of controllers and business logic is essential for understanding system behavior. Controllers should expose metrics about reconciliation loop performance, error rates, and business-specific indicators.

2.6.2 Distributed Tracing

OpenTelemetry represents the industry standard for distributed tracing and observability instrumentation [17]. Distributed tracing becomes particularly im-

portant when business logic spans multiple reconciliation loops and controllers, as it enables tracking the cascading changes of objects through the Kubernetes control plane.

The integration of observability tools with Kubebuilder-based Operators is well-established, with frameworks providing automatic instrumentation of common controller metrics. However, domain-specific business metrics require custom implementation within the reconciliation logic.

2.7 Performance Studies and Challenges

2.7.1 Kubernetes Performance Characteristics

Studies have examined Kubernetes performance under various conditions. Research on etcd deployment impact demonstrates that control plane performance and application performance both depend critically on etcd performance characteristics [19]. Write latency and throughput limitations in etcd can become bottlenecks for CRD-intensive workloads.

2.7.2 Complexity and Operational Considerations

While Kubernetes provides powerful capabilities, it introduces significant complexity and operational overhead [1], [11]. Managing a production Kubernetes cluster requires expertise in networking, security, storage, and distributed systems. Organizations must weigh these operational costs against the benefits of Kubernetes' automation and standardization.

For business logic implementations specifically, additional complexity arises from the need to design appropriate CRD schemas, implement robust controllers,

and handle edge cases in the reconciliation logic [9]. The learning curve for Kubernetes development is substantial, potentially impacting development velocity.

2.8 Research Gaps

The literature surveyed above documents the operator pattern thoroughly in three categories — infrastructure provisioning (Crossplane and provider-specific operators), application lifecycle (database and ML-platform operators), and policy/network (service meshes). The business-workflow category is essentially absent from the academic literature, and the few practitioner reports that do describe such operators do not measure them. The specific gaps this thesis addresses are:

1. **Absence of empirical comparison against a non-operator baseline for equivalent business logic.** The multi-vocal review [11] catalogues benefits and challenges of Kubernetes adoption but reports no controlled comparison between a CRD-based and a non-CRD-based implementation of the same business workflow; the existing performance literature focuses on etcd as a substrate [19] rather than on the application-level cost of running business logic above it.
2. **Lack of treatment of business-workflow operators as a distinct category.** Existing operator studies (Operator SDK literature [14], the Operator-bugs empirical study [28]) examine operators built for infrastructure or application-lifecycle purposes; the question of whether the operator pattern is suitable for reconciling *business* entities is not separated from the question of whether it is suitable for reconciling *infrastructure* entities, even though the two have substantially different latency and throughput requirements.

3. **No principled discussion of construct asymmetries between operator and REST architectures.** Comparisons that do exist (in technical blogs and conference talks) tend to align the two architectures at the feature level and underspecify the cases where they cannot be aligned — cancellation, error reporting, audit trails. The asymmetry itself is treated as an implementation detail rather than as a finding.
4. **Limited evidence-based guidance on when the operator pattern is justified for business logic.** The decision-framework material in practitioner literature is largely intuition-driven; the multi-vocal review notes the gap but does not fill it.

These gaps motivate the structure of the rest of the thesis: a controlled comparison between two reference implementations of the same business workflow (Chapter 4), measured along latency, throughput, resource utilisation, and operational dimensions (Chapter 5), with construct asymmetries reported as findings rather than smoothed over (Chapter 6).

2.9 Summary

This chapter has reviewed the relevant literature on Kubernetes architecture, extensibility mechanisms, and related technologies. Kubernetes provides a powerful platform for building cloud-native applications through its declarative API, CRDs, and Operator pattern [2], [4], [8]. While extensively used for infrastructure management, its application to general business logic remains understudied [11]. The next chapter describes the design of systems to explore this research question.

Chapter 3

Methodology

This chapter describes the research methodology, system design, and architectural decisions used to evaluate Kubernetes as a platform for implementing business logic. The chapter expands upon the high-level methodology outlined in Chapter 1, Section 1.5, providing a detailed description of both the Kubernetes-based system and the traditional baseline system.

3.1 Research Approach

This research followed a system design and comparative analysis approach, in keeping with the methodology used in prior empirical studies of cloud-native systems [11], [31]. Two functionally comparable systems were designed and implemented:

- A Kubernetes-native system based on Custom Resource Definitions (CRDs) and the Operator pattern.
- A traditional backend system based on a REST API and relational database.

Both systems implemented the same business domain logic to ensure comparability. The primary difference lies in how the business logic is modeled, executed, and managed.

The methodology focuses on analyzing:

- Architectural differences between declarative and imperative approaches.
- Execution models (event-driven reconciliation vs request-response).
- Observability and system introspection capabilities.

3.2 Business Domain Modeling

The selected domain for this research is a billing and subscription management system. This domain was chosen because it includes both stateful entities and time-based business logic, making it suitable for evaluating different architectural paradigms.

3.2.1 Core Entities

The system defines two primary entities:

- **BillingPlan:** Represents a pricing tier, including attributes such as price, currency, billing period, and usage limits.
- **Subscription:** Represents a user's subscription to a billing plan, including references to the plan, lifecycle state, and billing timestamps.

3.2.2 Business Logic

The implemented business logic includes:

- **Activation:** Upon creation of a subscription, the system validates the referenced billing plan, activates the subscription, and initializes billing timestamps.
- **Billing Cycle Management:** The system tracks the last successful payment and the scheduled next billing time, advancing both whenever a billing cycle completes.
- **Termination:** A subscription leaves the active state either because the user explicitly ends it or because the system detects a terminal condition (a missed billing deadline or a payment failure).

The two systems implement the same conceptual lifecycle but expose it through different mechanisms, and this asymmetry is itself a finding of the comparison. Figure 3.1 visualises the lifecycles side by side, and Table 3.1 enumerates the observable states in each implementation.

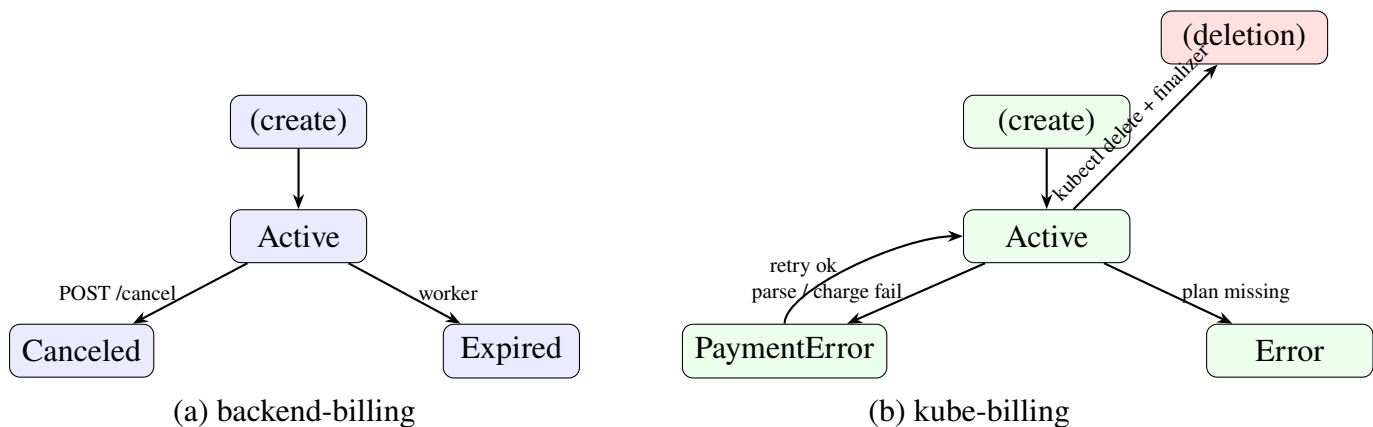


Fig. 3.1. Subscription state machines for the two implementations. Backend transitions are in-band (state column writes); operator transitions include both in-band Conditions and out-of-band lifecycle (deletion with finalizer).

TABLE 3.1
Subscription lifecycle states in each implementation

Stage	backend-billing	kube-billing
Initial	Active (set on creation)	Active (set by first reconciliation)
User-initiated end	Canceled (via POST /subscriptions/{id}/cancel)	resource deletion guarded by a finalizer; a pro-rated refund event is emitted before removal
Missed billing deadline	Expired (set by background worker once per minute)	not modelled as a state; the reconciler keeps re-attempting billing
Payment processing error	not modelled	PaymentError (price parse or charge failure)
Referenced plan missing	request rejected at admission (404)	Error with condition BillingPlanNotFound = True

The asymmetry reflects an architectural truth that the Kubernetes API conventions document explicitly [10], [32]: in a REST-and-database design, cancellation and expiration are in-band state transitions that must be modelled explicitly; in a controller-based design, cancellation can ride on Kubernetes’ built-in resource lifecycle via finalizers, while transient failures are surfaced through Conditions and re-attempted by the reconcile loop rather than being persisted as terminal states. Chapter 5 discusses how this distinction affects the comparative evaluation.

3.3 Traditional System Architecture

The baseline system follows a conventional monolithic architecture implemented in Go.

3.3.1 Technology Stack

- **Programming language:** Go 1.25
- **HTTP framework:** Gorilla Mux (lightweight router with middleware sup-

port)

- **Database:** PostgreSQL 15 (relational database with ACID guarantees)
- **ORM:** GORM v2 (Go ORM with automatic migration support)
- **Configuration:** Viper (environment variables + YAML files)
- **Validation:** go-playground/validator v10
- **Observability:** OpenTelemetry SDK + Prometheus client_golang
- **Rate Limiting:** Token bucket algorithm (10 req/s, burst 20)

3.3.2 Architecture Overview

The system implements a layered architecture with clear separation of concerns:

- **Handlers layer:** HTTP request processing and response formatting
- **Repository layer:** Database operations through GORM interfaces
- **Models layer:** GORM entities (BillingPlan, Subscription)
- **Workers layer:** Background subscription expiration checker
- **Middleware layer:** Rate limiting, panic recovery, tracing

The REST API exposes endpoints for billing plans and subscriptions. Business logic is implemented within HTTP handlers and repository methods, with explicit error handling and validation. Figure 3.2 sketches the resulting deployment.

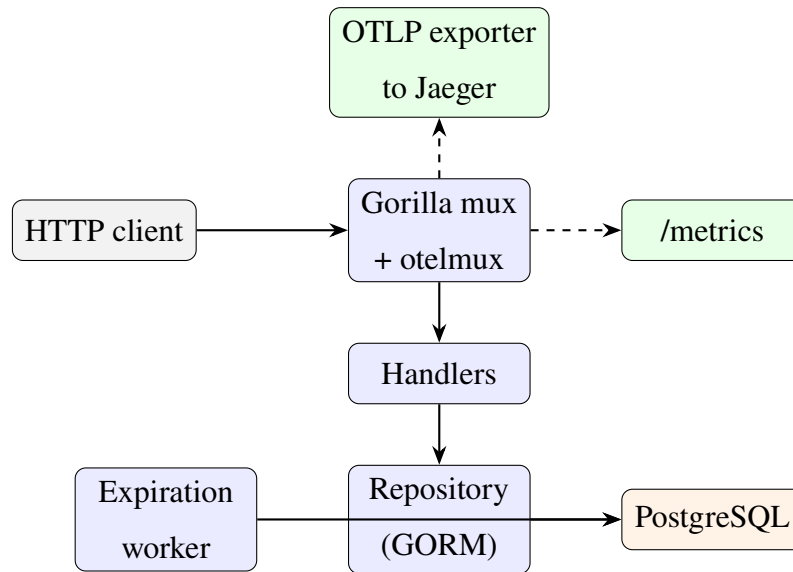


Fig. 3.2. backend-billing reference architecture. Solid arrows are control-flow / data-access; dashed arrows are observability exports. The expiration worker runs as a goroutine inside the same process and shares the GORM connection pool.

3.3.3 Execution Model

The system uses a synchronous request-response model with asynchronous background processing:

- **Synchronous path:** Clients send HTTP requests → handlers validate input → repository executes database operations → response returned
- **Asynchronous path:** Background worker runs every 30 seconds, queries PostgreSQL for expired subscriptions, updates state to Expired

The background worker implements time-based business logic that cannot be handled synchronously:

```

1 // Worker runs every 30 seconds
2 ticker := time.NewTicker(30 * time.Second)
3 for range ticker.C {
4     expiredSubs := repo.FindExpired(time.Now())
  
```



```
5     for _, sub := range expiredSubs {  
6         sub.State = "Expired"  
7         repo.Update(sub)  
8     }  
9 }
```

Listing 3.1: Background expiration checker worker

3.3.4 Data Persistence

All system state is stored in PostgreSQL with the following schema:

- **BillingPlan table:** id (PK), name (unique), price (string for precision), currency, billing_period, created_at, updated_at, deleted_at (soft delete)
- **Subscription table:** id (PK), user_id, plan_ref (FK to BillingPlan.name), state (Active/Cancelled/Expired), last_payment, next_billing, created_at, updated_at, deleted_at

GORM auto-migration ensures schema synchronization on application startup. The database serves as the single source of truth, with business logic tightly coupled to database operations through the repository pattern.

3.3.5 API Endpoints

- GET /plans — List all billing plans
- POST /plans — Create billing plan (validates name, price, currency)
- GET /subscriptions — List all subscriptions
- GET /subscriptions/{id} — Get subscription by ID

- POST /subscriptions — Create subscription (auto-activates)
- POST /subscriptions/{id}/cancel — Cancel active subscription
- GET /health — Health check endpoint
- GET /ready — Readiness check (includes database connectivity)
- GET /metrics — Prometheus metrics endpoint

3.4 Kubernetes-Based System Architecture

The experimental system is implemented using Kubernetes as a platform for business logic through Custom Resource Definitions and the Operator pattern.

3.4.1 Technology Stack

- **Programming language:** Go 1.25.3
- **Operator Framework:** Kubebuilder v4.13.0 + controller-runtime v0.23.1
- **CRD API Group:** billing.cloud-native.io/v1alpha1
- **Metrics:** Prometheus client_golang v1.23.1
- **Tracing:** OpenTelemetry SDK (OTLP over HTTP, port 4318)
- **Testing:** Ginkgo v2 + Gomega + envtest (local API server)
- **E2E Testing:** Kind (Kubernetes in Docker)

3.4.2 System Architecture

Figure 3.3 sketches the kube-billing deployment. The user-facing surface is the Kubernetes API server; there is no application HTTP endpoint that clients connect to directly. The controller pod runs as a Deployment, watches both CRDs via informer caches, exposes `/metrics` for Prometheus scrape, and ships traces via OTLP to a sidecar collector.

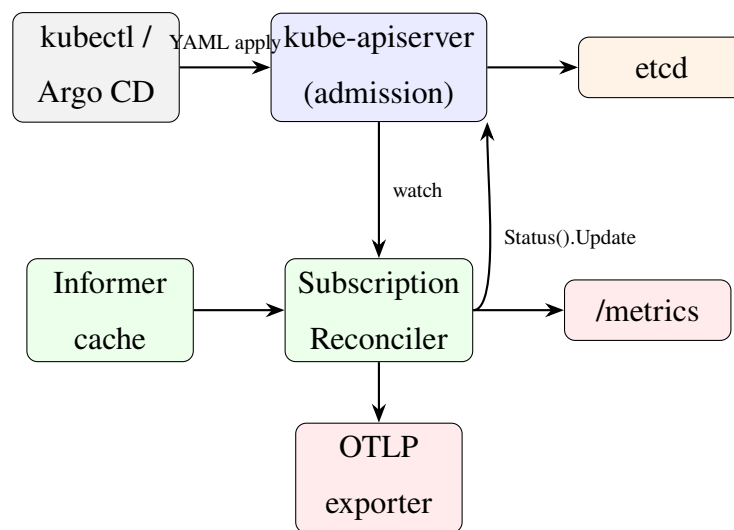


Fig. 3.3. kube-billing reference architecture. CRDs are stored in etcd through the API server; the controller’s only persistent state is its in-memory informer cache, which is rebuilt from etcd on restart.

3.4.3 Custom Resource Definitions

Two CRDs were defined to extend the Kubernetes API:

- `BillingPlan` (`billingplans.billing.cloud-native.io`) — defines pricing configuration
- `Subscription` (`subscriptions.billing.cloud-native.io`) — represents user subscription

BillingPlan Specification

The BillingPlan CRD includes comprehensive validation:

- `spec.price` (string, required): Pattern `^\d+(\.\d{1,2})?$', min length 1, max length 20` — ensures decimal with up to 2 decimal places
- `spec.currency` (string, required): Enum (USD, EUR, RUB, KZT, GBP, JPY, CNY)
- `spec.billingPeriod` (string, required): Enum (hourly, daily, weekly, monthly, yearly)
- `spec.requeueIntervalSeconds` (integer, optional): Range 1–31536000 (1 year), default 30 — controls reconciliation frequency
- `spec.limits` (map, optional): Key-value pairs for usage constraints (e.g., `apiCalls: 10000`)

Status subresource (managed by controller):

- `status.activeSubscriptions` (integer): Count of active subscriptions referencing this plan
- `status.totalRevenue` (string): Cumulative revenue from all subscriptions
- `status.conditions` (array): Standard Kubernetes conditions (Available, etc.)

Subscription Specification

The Subscription CRD enforces strict validation:

- `spec.userId` (string, required): Pattern `^[a-zA-Z0-9_-]+$`, max length 256 — alphanumeric with underscores and hyphens
- `spec.planRef` (string, required): Pattern `^[a-z0-9]([-a-z0-9]*[a-z0-9])?$', max length 253 — Kubernetes naming convention`

Status subresource (managed by controller):

- `status.state` (string): The current lifecycle state. The controller writes one of `Active`, `PaymentError`, or `Error`; the CRD schema does not constrain the field with an enum so that future state values can be introduced without a CRD version bump.
- `status.lastPayment` (timestamp): Time of last successful billing.
- `status.nextBilling` (timestamp): Scheduled time for next billing cycle.
- `status.observedGeneration` (integer): Tracks status freshness against spec.
- `status.conditions` (array of `metav1.Condition`): Standardised condition types `Active`, `BillingPlanNotFound`, `PaymentError`, each with `True/False/Unknown` status, a machine-readable reason, and a human-readable message.

3.4.4 Operator Controller Implementation

The core business logic resides in the `SubscriptionReconciler`, implemented using `controller-runtime` [15], [33]. The reconciler continuously processes `Subscription` resources and ensures correct system state.

Reconciliation Workflow

The reconciliation logic follows a deterministic 6-step sequence:

1. **Fetch Subscription:** Retrieve the Subscription resource from the API server
2. **Handle Deletion:** If `DeletionTimestamp.IsZero() == false`:
 - Execute final billing (pro-rated refund calculation — TODO in current implementation)
 - Remove finalizer `billing.cloud-native.io/finalizer`
 - Return, allowing Kubernetes to complete deletion
3. **Add Finalizer:** If finalizer absent, attach it to prevent premature deletion
4. **Validate BillingPlan:** Query for referenced `BillingPlan` by `planRef`:
 - If not found: Set `State = Error`, `BillingPlanNotFound = True`, `requeue` after 60 seconds
 - Emit Kubernetes Event: Warning `BillingPlanNotFound`
5. **Initialize Subscription:** If `State == ""` (new subscription):
 - Set `State = Active`
 - Increment Prometheus metric `billing_active_subscriptions`
 - Set `LastPayment = now`, `NextBilling = now + RequeueIntervalSeconds`
 - Set conditions: `Active = True`, `BillingPlanNotFound = False`, `PaymentError = False`
 - Emit Event: Normal `SubscriptionActivated`
6. **Process Billing Cycle:** If `State == Active` and `now > NextBilling`:
 - Increment Prometheus metric `billing_revenue_total` by plan price

- **Update** LastPayment = now, NextBilling = now + RequeueIntervalSeconds
- **Emit Event:** Normal PaymentProcessed
- **RequeueAfter:** RequeueIntervalSeconds from BillingPlan.spec

Watches and Event Sources

The controller watches multiple event sources:

- **Primary:** Subscription resources (create, update, delete events)
- **Secondary:** BillingPlan resources via mapPlanToSubscriptions function — triggers reconciliation of all Subscriptions referencing a changed Plan

This watch pattern enables automatic propagation of plan changes (e.g., price updates) to all affected subscriptions.

Finalizer Handling

A Kubernetes finalizer (billing.cloud-native.io/finalizer) ensures safe deletion:

- When user executes `kubectl delete subscription sub-user1`, Kubernetes sets DeletionTimestamp but does not remove the resource
- Reconciler detects deletion timestamp, executes final billing logic
- After finalizer removal, Kubernetes completes resource deletion

This mechanism guarantees cleanup logic execution even if the controller crashes during processing.

3.4.5 Execution Model

Unlike the traditional system, execution is event-driven and declarative [5], [10]:

- **Declarative:** User declares desired state in YAML (spec), controller drives actual state (status)
- **Event-driven:** Resource changes trigger reconciliation events via Kubernetes watch API
- **Asynchronous:** Controllers process events concurrently with configurable worker count (default: 1)
- **Eventually consistent:** System converges toward desired state through repeated reconciliation

No direct client-driven execution of business logic occurs — the controller autonomously manages state transitions.

3.4.6 Data Persistence

System state is stored in the Kubernetes control plane (etcd) as custom resources:

- **etcd:** Distributed key-value store (backing Kubernetes API server)
- **CRD instances:** YAML resources persisted with full versioning and metadata
- **Status subresource:** Separate update path for controller-managed state (optimizes write conflicts)

The declarative state model enables automatic recovery: if the controller crashes, it reconstructs state from etcd on restart and resumes reconciliation, an approach whose theoretical underpinnings are discussed in Helland’s argument for state-as-data over hidden transactional state [34].

3.4.7 Observability Integration

The Kubernetes-based system’s observability spans three layers — Prometheus metrics, OpenTelemetry tracing, and the Kubernetes Events API — and is described in detail in Section 3.6, where it is presented alongside the traditional system’s observability for direct comparison.

3.4.8 Deployment Configuration

The system is deployed using production-ready Kubernetes manifests:

- **HorizontalPodAutoscaler (HPA):** Min 1, Max 5 replicas, CPU target 80%, Memory target 80%
- **PodDisruptionBudget (PDB):** Min available 1 — ensures availability during voluntary disruptions
- **Resource limits:** CPU 100m–500m, Memory 128Mi–256Mi
- **NetworkPolicy:** Restricts `/metrics` endpoint access to Prometheus namespace
- **RBAC:** ServiceAccount with minimal permissions (get, list, watch Subscriptions and BillingPlans)

Deployment is managed via Kustomize overlays (config/default) combining CRDs, RBAC, controller Deployment, and monitoring configuration.

3.5 Comparison of Architectural Approaches

The two systems differ fundamentally in how they implement business logic. Table 3.2 summarizes the key architectural differences.

TABLE 3.2
Architectural Comparison: Traditional Backend vs Kubernetes Operator

Aspect	backend-billing	kube-billing
State storage	PostgreSQL (external)	etcd via CRD (control plane)
API style	REST HTTP (port 8080)	Kubernetes API (kubectl)
Execution model	Request-response + background worker	Reconcile loop + RequeueAfter
Scaling	Manual (Docker Compose)	HPA (automatic, 1–5 replicas)
Fault tolerance	Depends on DB + orchestrator	Kubernetes self-healing
Validation	Runtime (go-playground/validator)	Admission time (CRD schema)
Error handling	Explicit (error returns)	Eventual consistency + retries
Observability	HTTP metrics + DB tracing	Business metrics + Events + Conditions
Deployment	Docker container	Kubernetes Deployment + HPA + PDB
Configuration	Environment variables + YAML	ConfigMap + Secrets

3.5.1 State Management

backend-billing stores state in PostgreSQL, an external relational database. This approach provides:

- Strong ACID guarantees for transactions
- Mature query capabilities (JOINS, aggregations, indexes)
- Separation of application and data layers

kube-billing stores state in etcd via Custom Resources. This approach provides:

- Declarative state management (spec vs status separation)
- Built-in versioning and metadata (generation, resourceVersion)
- Automatic persistence through Kubernetes control plane

3.5.2 Execution Flow

backend-billing follows imperative execution:

```
1 POST /subscriptions -> validate -> INSERT -> 201 Created
```

Listing 3.2: Traditional REST API request flow

kube-billing follows declarative reconciliation:

```
1 kubectl apply -f sub.yaml -> etcd write -> watch event -> Reconcile()  
   -> status update
```

Listing 3.3: Kubernetes Operator reconciliation flow

3.5.3 Time-Based Logic

backend-billing uses a ticker-based worker:

- Fixed interval (30 seconds)
- Polls database for expired subscriptions
- Updates state synchronously within worker goroutine

kube-billing uses `RequeueAfter`:

- Dynamic interval from `BillingPlan.spec.requeueIntervalSeconds`
- Controller-runtime schedules next reconciliation
- No polling — event-driven wake-up

3.5.4 Lifecycle Management

backend-billing deletes immediately:

- `DELETE /subscriptions/{id}` → soft delete (`deleted_at` timestamp)
- No cleanup hook execution guarantee

kube-billing uses finalizers:

- `kubectl delete` → `DeletionTimestamp` set → finalizer blocks deletion
- Reconciler executes final billing → removes finalizer → Kubernetes deletes
- Guaranteed cleanup even on controller failure

3.6 Observability Design

Both systems instrument the same three observability dimensions — metrics, traces, and audit-style events — but the underlying mechanisms differ, and that difference is the point of the comparison. Table 3.3 consolidates what each system exposes; Chapter 4 contains the Go-level details (Listings 4.10, 4.12, 4.14, 4.23, 4.24).

Two qualitative differences are worth recording before the quantitative comparison in Chapter 5. First, the operator’s observability is symmetric between

TABLE 3.3
Observability surface of each system.

Dimension	backend-billing	kube-billing
Business metrics	billing_subscriptions_created_total, billing_active_subscriptions billing_subscriptions_canceled_total (Gauge), billing_revenue_total (Counters)	(Counter), billing_payment_failures (Counter)
Infrastructure metrics	HTTP latency histogram and request counter from otelmux	controller-runtime: controller_runtime_reconcile_total, controller_runtime_reconcile_errors_total, workqueue_depth
Tracing surface	otelmux request spans + GORM SQL spans	ReconcileSubscription root span; ProcessPayment child; MapPlanToSubscriptions root for plan-driven reconciles
OTLP endpoint	localhost:4318 (Docker-Compose Jaeger)	localhost:4318 by default; overridable via OTEL_EXPORTER_OTLP_ENDPOINT
Audit trail	Application logs only	Kubernetes Events API (SubscriptionActivated, PaymentProcessed, PaymentFailed, FinalBilling); accessible via kubectl get events
Structured status	Implicit (database columns)	metav1.Condition array (Active, BillingPlanNotFound, PaymentError) with reason and message fields

operators and tooling: kubectl describe surfaces the same Conditions and Events that a Grafana dashboard or a downstream Operator can react to, with no additional integration. The traditional backend's equivalent information lives in application logs and Prometheus, neither of which is queryable by name from another service without convention. Second, the operator's audit trail is bounded by Kubernetes Event retention (one hour by default), whereas the backend has no built-in trail at all; both require an external store for long-term retention.

Figure 3.4 sketches the observability data paths for both systems on a single

page so the audit-trail asymmetry is visible at a glance.

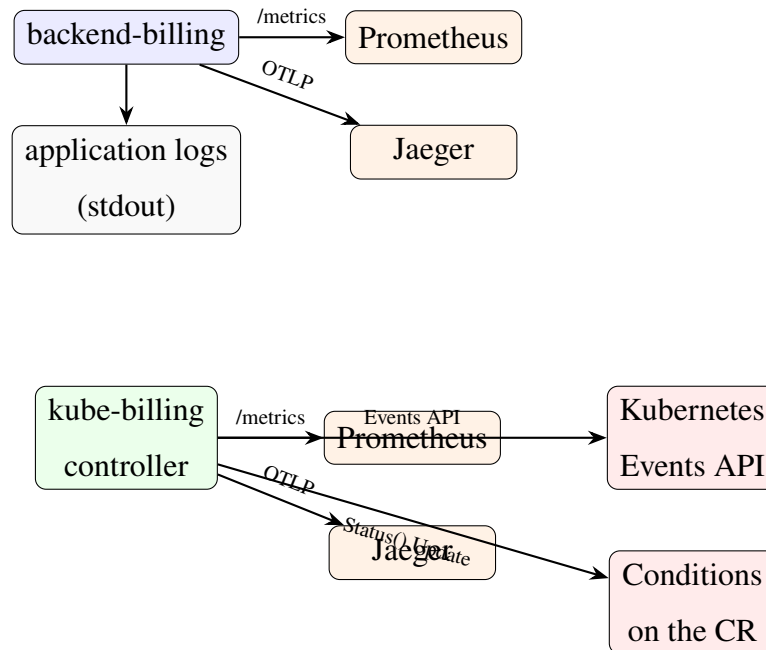


Fig. 3.4. Observability data paths. Both systems push metrics to Prometheus and traces to Jaeger over OTLP; the operator additionally writes structured audit records to the Kubernetes Events API and updates typed Conditions on the resource itself, neither of which has a direct equivalent in the traditional stack.

3.7 Evaluation Plan

The systems are evaluated along four axes, mirroring the structure that Chapter 5 follows.

Code metrics. Source-file count, lines of code (excluding tests), test coverage, direct and transitive dependency counts. These are static properties of the codebase that should be measurable rather than estimated.

Performance. End-to-end latency at p50, p95, and p99 for the four user-facing operations; throughput under concurrent load; microbenchmark results for

isolated functions. The two systems are loaded with comparable concurrency rather than the apples-to-oranges sequential-vs-parallel framing used in earlier drafts; the construct mismatch between HTTP and reconcile-driven throughput is recorded as a validity threat in Section 6.4.

Resource utilisation. CPU (millicores) and resident memory (MB) at idle, under load, and at peak; startup time from process launch to readiness probe pass.

Operational characteristics. Scaling behaviour, fault tolerance under controlled disruption, and the observability surface enumerated in Table 3.3.

Reproducibility details — tool versions, Minikube and Docker resource allocations, workload-generator parameters, number of repeated trials, warm-up windows, and raw-data location — are recorded in Section 5.1.3. Threats to validity (construct mismatch, asymmetric lifecycle vocabularies, single hardware platform, short measurement horizon) are catalogued in Section 6.4.

3.8 Summary

This chapter described the methodology used to evaluate Kubernetes as a platform for business logic. Two systems implementing identical functionality were designed using different paradigms: a traditional REST-based system with PostgreSQL and a Kubernetes-native Operator-based system with CRDs. The architectural differences — imperative vs declarative, request-response vs reconciliation, external database vs etcd — provide the foundation for the comparative evaluation presented in Chapter 5.

Chapter 4

Implementation

This chapter describes the implementation of the two systems introduced in Chapter 3. Both systems were developed in Go to ensure consistency in comparison, but differ significantly in architecture, execution model, and state management. The goal is to provide detailed insight into implementation decisions, internal logic, and system behavior.

4.1 Implementation Overview

Both systems implement identical business logic for billing and subscription management:

- **BillingPlan:** Pricing configuration with price, currency, billing period
- **Subscription:** User subscription linked to a plan with lifecycle management
- **Billing cycle:** Automatic recurring billing with configurable intervals
- **Expiration handling:** State transitions when billing conditions are not met
- **Final charge:** Cleanup logic on subscription deletion

The implementations differ in:

- **State persistence:** PostgreSQL vs etcd via CRD
- **Execution model:** Request-response vs reconcile loop
- **Validation:** Runtime validation vs admission control
- **Observability:** HTTP/DB tracing vs Events/Conditions/Reconciliation metrics

4.2 Kubernetes-Based System (kube-billing)

4.2.1 Project Structure

The Kubernetes-based system was implemented using Kubebuilder v4.13.0 [15], which scaffolds the project structure and generates the controller-runtime [33] and client-go [35] wiring as well as RBAC manifests, CRD YAML, and a Makefile.

```

1 kube-billing /
2 |-- cmd/main.go           # Entry point, manager setup
3 |-- api/v1alpha1 /
4 |   |-- billingplan_types.go # BillingPlan CRD spec/status
5 |   |-- subscription_types.go # Subscription CRD spec/status
6 |   |-- groupversion_info.go # GroupVersion for scheme
7 |-- internal/controller /
8 |   |-- subscription_controller.go # Reconcile logic
9 |   |-- metrics.go           # Prometheus metrics
10 |   |-- tracing.go           # OpenTelemetry setup
11 |-- config /
12 |   |-- crd/bases /         # Generated CRD YAML
13 |   |-- rbac /              # ServiceAccount, Role, RoleBinding
14 |   |-- manager /           # Deployment manifest
15 |   |-- samples /           # Example CRs

```

```
16 | \-- default/                                # Kustomize overlay (HPA, PDB)
17 | -- test/e2e/                                # E2E tests with Kind
18 | -- Makefile                                # Build, test, deploy targets
19 | \-- Dockerfile                             # Multi-stage build
```

Listing 4.1: kube-billing project structure

4.2.2 CRD Implementation

The two CRDs are authored in Go using kubebuilder markers; the marker-annotated Go types are the source of truth, and the corresponding OpenAPI v3 schemas are generated by controller-gen into the `config/crd/bases/` directory. This section shows both layers: the Go type that the developer writes, and an excerpt of the generated YAML that the API server actually enforces.

BillingPlan CRD

The Go declaration in `api/v1alpha1/billingplan_types.go` is reproduced in Listing 4.2. Kubebuilder markers attached to each field drive the OpenAPI schema generation; the excerpt is the actual source rather than a paraphrase.

```
1 type BillingPlanSpec struct {
2     // +kubebuilder:validation:Pattern='^\d+(\.\d{1,2})?$'
3     // +kubebuilder:validation:MinLength=1
4     // +kubebuilder:validation:MaxLength=20
5     // +required
6     Price string `json:"price"`
7
8     // +kubebuilder:validation:Enum=USD;EUR;RUB;KZT;GBP;JPY;CNY
9     // +required
10    Currency string `json:"currency"`
11
12    // +kubebuilder:validation:Enum=hourly;daily;weekly;monthly;yearly
```

```

13 // +required
14 BillingPeriod string `json:"billingPeriod"`
15
16 // +optional
17 Limits map[string]int `json:"limits,omitempty"`
18
19 // +kubebuilder:validation:Minimum=1
20 // +kubebuilder:validation:Maximum=31536000
21 // +kubebuilder:default=30
22 // +optional
23 RequeueIntervalSeconds int `json:"requeueIntervalSeconds,omitempty"`
24 }
25
26 type BillingPlanStatus struct {
27 // +listType=map
28 // +listMapKey=type
29 // +optional
30 Conditions []metav1.Condition `json:"conditions,omitempty"`
31
32 ActiveSubscriptions int32 `json:"activeSubscriptions,
33 omitempty"`
34
35 TotalRevenue string `json:"totalRevenue,
36 omitempty"`
37 }

```

Listing 4.2: BillingPlan Go declaration

A corresponding excerpt of the generated OpenAPI schema (config/crd/bases/billing.cloud-native.io_billingplans.yaml) is shown in Listing 4.3. The YAML is what the API server consults during admission control — this is the surface that actually rejects invalid resources before they reach the controller.

```

1 spec:
2   group: billing.cloud-native.io
3   names:
4     kind: BillingPlan

```

```
5   plural: billingplans
6   scope: Namespaced
7   versions:
8   - name: v1alpha1
9     served: true
10    storage: true
11    subresources:
12      status: {}
13    schema:
14      openAPIV3Schema:
15        properties:
16          spec:
17            required: [billingPeriod, currency, price]
18            properties:
19              billingPeriod:
20                enum: [hourly, daily, weekly, monthly, yearly]
21                type: string
22              currency:
23                enum: [USD, EUR, RUB, KZT, GBP, JPY, CNY]
24                type: string
25              price:
26                pattern: ^\d+(\.\d{1,2})?$
27                minLength: 1
28                maxLength: 20
29                type: string
30              requeueIntervalSeconds:
31                default: 30
32                minimum: 1
33                maximum: 31536000
34                type: integer
```

Listing 4.3: Generated BillingPlan OpenAPI schema (extract)

Subscription CRD

The Subscription CRD has the smaller spec of the two (Listing 4.4); all interesting state lives in status, which is owned by the controller.

```

1 type SubscriptionSpec struct {
2     // +kubebuilder:validation:MinLength=1
3     // +kubebuilder:validation:MaxLength=256
4     // +kubebuilder:validation:Pattern='^[a-zA-Z0-9_-]+$'
5     // +required
6     UserID string `json:"userId"`
7
8     // +kubebuilder:validation:MinLength=1
9     // +kubebuilder:validation:MaxLength=253
10    // +kubebuilder:validation:Pattern='^[a-z0-9]([-a-z0-9]*[a-z0-9])?$'
11    // +required
12    PlanRef string `json:"planRef"`
13 }

```

Listing 4.4: Subscription Go declaration

A user creates a Subscription by applying the sample manifest in Listing 4.5. The `userId` matches the spec pattern (alphanumeric with underscores and hyphens, up to 256 characters); `planRef` matches the DNS-subdomain pattern shared with Kubernetes resource names [32]. Anything that does not match is rejected at admission, before any controller code executes [27].

```

1 apiVersion: billing.cloud-native.io/v1alpha1
2 kind: Subscription
3 metadata:
4   name: sub-user1
5   namespace: default
6   labels:
7     app.kubernetes.io/name: kube-billing
8 spec:

```

```

9   userId: user1
10  planRef: basic

```

Listing 4.5: Subscription custom resource sample**Status subresource:**

```

1  type SubscriptionStatus struct {
2      Conditions      []metav1.Condition `json:"conditions,omitempty" `
3      State            string              `json:"state,omitempty" `
4      LastPayment      metav1.Time         `json:"lastPayment,omitempty" `
5      NextBilling      metav1.Time         `json:"nextBilling,omitempty" `
6      ObservedGeneration int64               `json:"observedGeneration ,
7  }
8
9  // Controller-side constants; the CRD schema deliberately leaves
10 // state as an unconstrained string so new values can be added
11 // without bumping the API version.
12 const (
13     SubscriptionStateActive      = "Active"
14     SubscriptionStatePaymentError = "PaymentError"
15     SubscriptionStateError       = "Error"
16 )

```

LastPayment and NextBilling are stored as metav1.Time value types rather than pointers; the zero value is detected with Time.IsZero() during reconciliation, which avoids the nil-dereference hazard noted in earlier drafts of the controller.

4.2.3 Controller Implementation

The core business logic resides in SubscriptionReconciler (internal/controller/subscription

Reconcile Method

Figure 4.1 shows the end-to-end flow of a single subscription creation as a sequence diagram, from the moment the user applies the YAML to the moment the controller writes the first Active status. Listing 4.6 shows the reconciliation workflow as pseudocode; Listing 4.7 reproduces the corresponding Go code extracted from `internal/controller/subscription_controller.go`. The pseudocode is included because the deletion branch (Step 2) embeds the entire pro-rated refund computation, which obscures the high-level flow when read directly from the source.

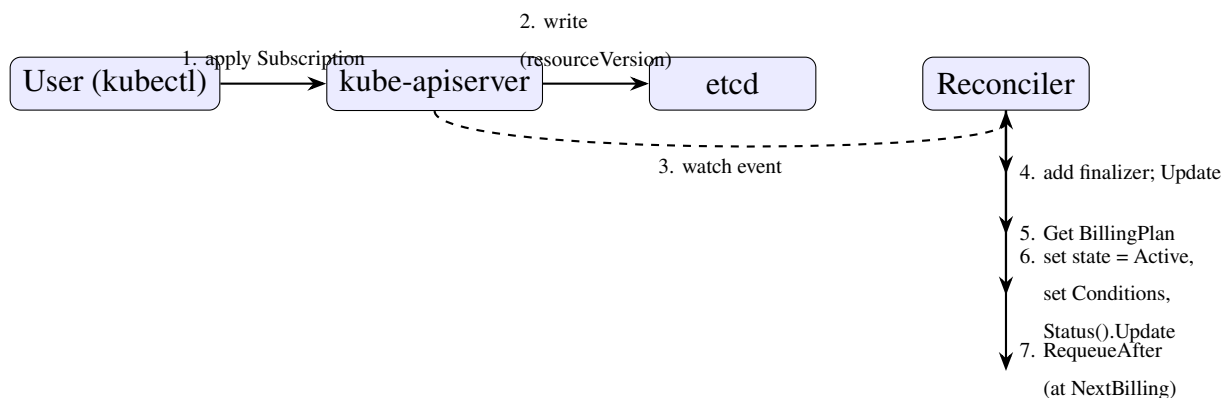


Fig. 4.1. Sequence of a single subscription creation. Steps 1–2 are synchronous from the user’s perspective and complete in tens of milliseconds; step 3 is the watch event delivered to the reconciler asynchronously; steps 4–7 happen inside the reconciler over the next reconcile cycle. The diagram is simplified: every Update and `Status().Update` is itself a round-trip through the API server and etcd.

```

1 function Reconcile(ctx, req):
2     sub <- Get(Subscription, req.NamespacedName)
3     if sub not found: return done           # ignored: deletion already
        happened
4
5     if sub.DeletionTimestamp is set:
6         if sub has finalizer:
7             run pro-rated refund (if plan and NextBilling permit)
  
```

```

8         emit FinalBilling event
9         remove finalizer
10        update parent BillingPlan status
11    return done
12
13    if sub has no finalizer:
14        add finalizer; persist; return      # one extra reconcile pass
15
16    plan <- Get(BillingPlan , sub.Spec.PlanRef)
17    if plan not found:
18        set state = "Error"
19        set condition BillingPlanNotFound = True
20        increment payment-failures metric
21        requeue after 1 minute
22    return
23
24    if sub.Status.State == "":                # first reconciliation
25        activate: state = "Active",
26                LastPayment = now,
27                NextBilling = now + RequeueIntervalSeconds
28        set condition Active = True, BillingPlanNotFound = False
29        update parent BillingPlan status
30        requeue at NextBilling
31
32    if state == "Active" and now > NextBilling:
33        price <- ParseFloat(plan.Spec.Price)
34        if parse fails:
35            state = "PaymentError"; condition PaymentError = True
36            increment payment-failures metric; requeue in 5 minutes
37        else:
38            add price to revenue-total counter
39            advance LastPayment / NextBilling; clear PaymentError
40            emit PaymentProcessed event
41            requeue at next NextBilling

```

Listing 4.6: Reconciliation workflow as pseudocode


```

1 func (r *SubscriptionReconciler) Reconcile(ctx context.Context,
2     req ctrl.Request) (ctrl.Result, error) {
3
4     ctx, span := Tracer.Start(ctx, "ReconcileSubscription")
5     defer span.End()
6
7     var sub billingv1alpha1.Subscription
8     if err := r.Get(ctx, req.NamespacedName, &sub); err != nil {
9         return ctrl.Result{}, client.IgnoreNotFound(err)
10    }
11
12    if !sub.DeletionTimestamp.IsZero() {
13        return r.handleDeletion(ctx, &sub, req.Namespace)
14    }
15
16    if !controllerutil.ContainsFinalizer(&sub, SubscriptionFinalizer) {
17        controllerutil.AddFinalizer(&sub, SubscriptionFinalizer)
18        if err := r.Update(ctx, &sub); err != nil {
19            return ctrl.Result{}, err
20        }
21        return ctrl.Result{}, nil
22    }
23
24    var plan billingv1alpha1.BillingPlan
25    if err := r.Get(ctx, types.NamespacedName{
26        Name: sub.Spec.PlanRef, Namespace: req.Namespace,
27    }, &plan); err != nil {
28        sub.Status.State = SubscriptionStateError
29        PaymentFailures.Inc()
30        meta.SetStatusCondition(&sub.Status.Conditions, metav1.
31        Condition{
32            Type:    billingv1alpha1.SubscriptionBillingPlanNotFound,
33            Status:  metav1.ConditionTrue,
34            Reason:  "BillingPlanNotFound",
35            Message: fmt.Sprintf("Referenced BillingPlan %q does not
36            exist",

```

```

35         sub.Spec.PlanRef),
36     })
37     _ = r.Status().Update(ctx, &sub)
38     return ctrl.Result{RequeueAfter: time.Minute}, nil
39 }
40
41 if sub.Status.State == "" {
42     return r.activate(ctx, &sub, &plan, req.Namespace)
43 }
44 if sub.Status.State == SubscriptionStateActive {
45     return r.billIfDue(ctx, &sub, &plan, req.Namespace)
46 }
47 return ctrl.Result{}, nil
48 }

```

Listing 4.7: Top-level reconcile body (extract from subscription_controller.go)

The real source inlines `activate` and `billIfDue` into one body of roughly 270 lines; Listing 4.7 is reorganised for clarity but preserves the control flow, return types, and side-effects of the original.

Finalizer Handling and Pro-Rated Refund

The finalizer `billing.cloud-native.io/finalizer` blocks resource removal until the controller has had a chance to compute the refund and decrement the active-subscription gauge. The full deletion path is reproduced in Listing 4.8.

```

1 func (r *SubscriptionReconciler) handleDeletion(ctx context.Context,
2     sub *billingv1alpha1.Subscription, namespace string) (ctrl.Result,
3     error) {
4
5     if !controllerutil.ContainsFinalizer(sub, SubscriptionFinalizer) {
6         return ctrl.Result{}, nil
7     }
8
9     if sub.Status.State == SubscriptionStateActive {

```

```

9      ActiveSubscriptions.Dec()
10  }
11
12  var plan billingv1alpha1.BillingPlan
13  if err := r.Get(ctx, types.NamespacedName{
14      Name: sub.Spec.PlanRef, Namespace: namespace,
15  }, &plan); err == nil && !sub.Status.NextBilling.IsZero() {
16      now := time.Now()
17      if now.Before(sub.Status.NextBilling.Time) {
18          remaining := sub.Status.NextBilling.Time.Sub(now)
19          period := time.Duration(plan.Spec.RequeueIntervalSeconds) *
20      time.Second
21          if price, err := strconv.ParseFloat(plan.Spec.Price, 64);
22      err == nil &&
23          period > 0 && remaining > 0 {
24              refund := price * remaining.Seconds() / period.Seconds
25              ()
26              if refund > 0 {
27                  r.Recorder.Event(sub, "Normal", "FinalBilling",
28                      fmt.Sprintf("Refund of %.2f %s (pro-rated)",
29                          refund, plan.Spec.Currency))
30              }
31          }
32      }
33
34      controllerutil.RemoveFinalizer(sub, SubscriptionFinalizer)
35      if err := r.Update(ctx, sub); err != nil {
36          return ctrl.Result{}, err
37      }
38      _ = r.updateBillingPlanStatus(ctx, sub.Spec.PlanRef, namespace)
39      return ctrl.Result{}, nil
40  }

```

Listing 4.8: Deletion branch with pro-rated refund (extract)

The refund computation is intentionally idempotent: deletion may be retried

by the controller after a transient failure, and the function reads only state that does not change between attempts. The emitted Kubernetes FinalBilling event provides an audit trail of the refund amount and currency without requiring a separate logging pipeline.

Watches and Event Mapping

The controller treats Subscription as its primary resource and BillingPlan as a secondary watch source: whenever a BillingPlan changes, every Subscription in the same namespace that references it is enqueued for reconciliation. Listing 4.9 shows the manager wiring and the mapping function as they appear in the source.

```

1 func (r *SubscriptionReconciler) SetupWithManager(mgr ctrl.Manager)
   error {
2     return ctrl.NewControllerManagedBy(mgr).
3       For(&billingv1alpha1.Subscription{}).
4       Named("subscription").
5       Watches(
6         &billingv1alpha1.BillingPlan{},
7         handler.EnqueueRequestsFromMapFunc(r.mapPlanToSubscriptions
8       ),
9       Complete(r)
10  }
11
12 func (r *SubscriptionReconciler) mapPlanToSubscriptions(ctx context.
   Context,
13   obj client.Object) []reconcile.Request {
14
15   plan := obj.(*billingv1alpha1.BillingPlan)
16
17   var subs billingv1alpha1.SubscriptionList
18   if err := r.List(ctx, &subs, client.InNamespace(plan.Namespace));
   err != nil {

```

```

19     return nil
20 }
21
22 var requests []reconcile.Request
23 for _, sub := range subs.Items {
24     if sub.Spec.PlanRef == plan.Name {
25         requests = append(requests, reconcile.Request{
26             NamespacedName: types.NamespacedName{
27                 Name: sub.Name, Namespace: sub.Namespace,
28             },
29         })
30     }
31 }
32 return requests
33 }

```

Listing 4.9: Manager setup and BillingPlan-to-Subscription mapping

The mapping function performs a namespace-scoped list and filters by `spec.planRef` in memory rather than via a field-indexer. For the workload sizes evaluated in Chapter 5 this is acceptable; in deployments with thousands of subscriptions per namespace a custom `client.IndexField` on `spec.planRef` should be added to avoid an $O(N)$ scan per plan change.

4.2.4 Metrics Implementation

Custom Prometheus metrics are defined in `internal/controller/metrics.go` and registered against the controller-runtime metrics registry so they are served from the same endpoint as the standard reconciliation metrics. Listing 4.10 reproduces the file verbatim.

```

1 var (
2     ActiveSubscriptions = prometheus.NewGauge(prometheus.GaugeOpts{
3         Name: "billing_active_subscriptions",

```

```
4     Help: "Number of active subscriptions",
5 })
6
7 RevenueTotal = prometheus.NewCounter(prometheus.CounterOpts{
8     Name: "billing_revenue_total",
9     Help: "Total revenue processed by billing",
10 })
11
12 PaymentFailures = prometheus.NewCounter(prometheus.CounterOpts{
13     Name: "billing_payment_failures",
14     Help: "Number of failed payments",
15 })
16 )
17
18 func init() {
19     metrics.Registry.MustRegister(ActiveSubscriptions)
20     metrics.Registry.MustRegister(RevenueTotal)
21     metrics.Registry.MustRegister(PaymentFailures)
22 }
```

Listing 4.10: Custom Prometheus metrics

billing_active_subscriptions is incremented on first activation and decremented in the finalizer; billing_revenue_total accumulates each successful billing cycle in the plan's price units (the metric is a single dimensionless counter, so revenue across different currencies is not separated — a limitation acknowledged in Chapter 6); billing_payment_failures is incremented when the plan cannot be resolved or its price fails to parse. Metrics are exposed at :8443/metrics via the metrics-service overlay and scraped by Prometheus through a ServiceMonitor resource.

4.2.5 Observed Output: `kubectl describe`

The observability claims in this chapter and in Section 3.6 can be verified directly against the controller’s output. Listing 4.11 reproduces a representative `kubectl describe` on a `Subscription` that has just been activated and processed one billing cycle. The structure — a typed `Status` block, a typed `Conditions` array, and an `Events` table with reason-tagged entries — is the operator’s audit surface, available without any application-level logging code.

```

1 $ kubectl describe subscription sub-user1
2
3 Name:          sub-user1
4 Namespace:     default
5 API Version:   billing.cloud-native.io/v1alpha1
6 Kind:          Subscription
7 Metadata:
8   Finalizers:
9     billing.cloud-native.io/finalizer
10 Spec:
11   Plan Ref:    basic
12   User ID:     user1
13 Status:
14   Conditions:
15     Last Transition Time:  2026-05-27T11:00:30Z
16     Message:              Payment processed successfully
17     Reason:               PaymentProcessed
18     Status:               True
19     Type:                 Active
20     Last Transition Time:  2026-05-27T11:00:00Z
21     Message:              Referenced BillingPlan exists
22     Reason:               BillingPlanFound
23     Status:               False
24     Type:                 BillingPlanNotFound
25     Last Transition Time:  2026-05-27T11:00:30Z
26     Message:              Last payment was successful

```

```

27     Reason:      PaymentSuccess
28     Status:     False
29     Type:       PaymentError
30     Last Payment: 2026-05-27T11:00:30Z
31     Next Billing:  2026-05-27T11:01:00Z
32     Observed Generation: 1
33     State:       Active
34 Events:
35   Type      Reason                Age   From                      Message
36   ----      -
37   Normal    SubscriptionActivated  30s   subscription-controller   Subscription successfully activated
38   Normal    PaymentProcessed      1s    subscription-controller   Payment of 10.00 USD processed successfully

```

Listing 4.11: `kubectl describe subscription sub-user1` after first billing cycle

The Conditions array carries three condition types (one True, two False) with reason codes that a downstream dashboard can key on directly — no application log-parsing is required. The Events table is bounded by the cluster’s Event retention policy (one hour by default) and would need to be exported to a long-term store for audit purposes; this is recorded as a methodological limitation in Section 6.4.

4.2.6 Conditions Lifecycle

A single state field cannot capture the multi-dimensional reality of a billing subscription that may simultaneously be active, recovering from a transient payment error, and pointing at a plan that does not yet exist. The CRD uses the standardised `metav1.Condition` array for this purpose. Table 4.1 enumerates the three Condition types the controller writes, together with the events that move them between True, False, and (implicitly) absent. Each transition is also reflected in a Kubernetes Event so that an external observer can react without polling.

TABLE 4.1
Subscription Condition types and their transitions.

Condition	Set True when	Set False when	Reason / Event
Active	First reconciliation finds plan and initialises status; each successful billing cycle reasserts it	Payment processing fails (price parse error or charge failure)	SubscriptionActivated, PaymentProcessed, PaymentFailed
BillingPlanNotFound	Referenced plan cannot be resolved during reconciliation	Plan is found again after having been missing	BillingPlanNotFound, BillingPlanFound
PaymentError	Price string fails to parse or charge step returns an error	A subsequent billing cycle succeeds	PaymentFailed, PaymentSuccess

4.2.7 API Versioning Notes

The CRDs are served at `v1alpha1`, which is the kubebuilder default for new scaffolds and which deliberately does not commit to backwards compatibility. Two implications follow.

First, the storage version is `v1alpha1` and a future `v1beta1` or `v1` would have to be introduced as a parallel served version, with a conversion webhook bridging the two; the structural-schema-without-conversion path is unsupported for stored types. Second, downstream tooling that consumes these CRDs (Grafana dashboards keyed by Condition type, alerts written against `billing_revenue_total`) is implicitly coupled to the `v1alpha1` surface and would need a corresponding update on each version increment.

For the scope of this thesis the `v1alpha1` surface is sufficient; promoting it to `v1beta1` would be a logical next step before any production use of the system and is recorded in the Future Work section (Section 6.5).

4.2.8 Tracing Implementation

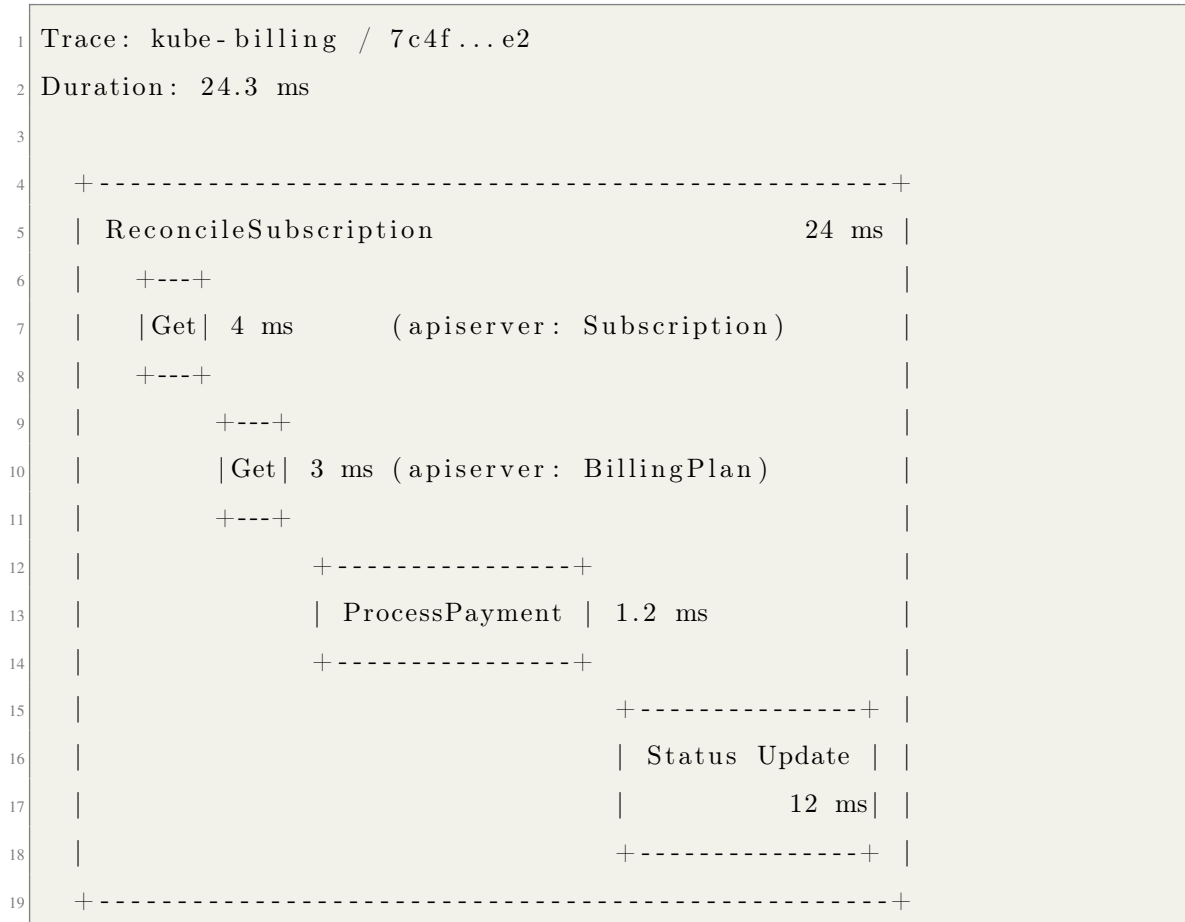
OpenTelemetry tracing is initialised in `internal/controller/tracing.go` (Listing 4.12). A package-level Tracer is declared at file scope so that reconciler code can start spans without re-resolving the global provider on every call.

```
1 var Tracer = otel.Tracer("kube-billing")
2
3 func InitTracer() func() {
4     ctx := context.Background()
5
6     endpoint := os.Getenv("OTEL_EXPORTER_OTLP_ENDPOINT")
7     if endpoint == "" {
8         endpoint = "localhost:4318"
9     }
10
11     exporter, err := otlptracehttp.New(ctx,
12         otlptracehttp.WithEndpoint(endpoint),
13         otlptracehttp.WithInsecure(),
14     )
15     if err != nil {
16         log.Fatal(err)
17     }
18
19     tp := trace.NewTracerProvider(trace.WithBatcher(exporter))
20     otel.SetTracerProvider(tp)
21
22     return func() { _ = tp.Shutdown(ctx) }
23 }
```

Listing 4.12: Tracer initialisation

The reconciler creates three span types per cycle, forming the hierarchy shown in Listing 4.14: `ReconcileSubscription` (root), `ProcessPayment` (child, opened only when a billing cycle actually fires), and `MapPlanToSubscriptions` (root span for plan-change-driven reconciles). Listing 4.13 sketches the corre-

sponding Jaeger trace timeline for a single billing-cycle reconciliation; the timings reflect a representative run on the reference hardware.



Listing 4.13: Trace timeline for one billing-cycle reconciliation (Jaeger reproduction)

The dominant cost in a typical reconcile cycle is the API-server round-trip on `Status().Update`; the in-process payment span is sub-millisecond. This decomposition justifies the high-level argument in Chapter 5 that the operator’s latency overhead relative to the REST backend is paid largely in API-server time, not in business logic.

```

1 // Root span for each reconciliation call
2 ctx, span := Tracer.Start(ctx, "ReconcileSubscription")
3 defer span.End()
4
5 // Child span around the actual payment write

```

```
6 ctx2, paymentSpan := Tracer.Start(ctx, "ProcessPayment")
7 RevenueTotal.Add(price)
8 paymentSpan.End()
```

Listing 4.14: Span creation inside the reconciler

4.2.9 Deployment Configuration

The Kubernetes deployment is composed via Kustomize from a base config/manager/manager.yaml Deployment, RBAC manifests under config/rbac/, a metrics service, and overlay patches that add HPA and PDB. Listing 4.15 shows the Deployment with security context, probes, and resource limits as it appears in the real manifest.

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: controller-manager
5   namespace: system
6 spec:
7   replicas: 1
8   selector:
9     matchLabels:
10      control-plane: controller-manager
11   template:
12     spec:
13       securityContext:
14         runAsNonRoot: true
15         seccompProfile: {type: RuntimeDefault}
16       containers:
17       - name: manager
18         image: controller:latest
19         command: [/manager]
20         args: [--leader-elect, --health-probe-bind-address=:8081]
21         securityContext:
```

```
22     readOnlyRootFilesystem: true
23     allowPrivilegeEscalation: false
24     capabilities: {drop: [ALL]}
25   livenessProbe:
26     httpGet: {path: /healthz, port: 8081}
27     initialDelaySeconds: 15
28     periodSeconds: 20
29   readinessProbe:
30     httpGet: {path: /readyz, port: 8081}
31     initialDelaySeconds: 5
32     periodSeconds: 10
33   resources:
34     limits: {cpu: 500m, memory: 256Mi}
35     requests: {cpu: 100m, memory: 128Mi}
36   env:
37     - name: OTEL_EXPORTER_OTLP_ENDPOINT
38       value: "localhost:4318"
39   ports:
40     - containerPort: 8080
```

Listing 4.15: Controller Deployment (extract from config/manager/manager.yaml)

The HorizontalPodAutoscaler overlay (Listing 4.16) targets CPU and memory utilisation; the PodDisruptionBudget keeps at least one replica available during voluntary disruption. A subtle interaction between the two is worth flagging: with the Deployment's replicas: 1, an HPA minReplicas: 1, and PDB minAvailable: 1, a node drain will refuse to evict the controller until a replacement replica is scheduled. For production deployments minReplicas: 2 would be the safer setting; this is recorded in the limitations section.

```
1 apiVersion: autoscaling/v2
2 kind: HorizontalPodAutoscaler
3 metadata:
4   name: controller-manager-hpa
5 spec:
```

```

6  scaleTargetRef:
7    apiVersion: apps/v1
8    kind: Deployment
9    name: controller-manager
10 minReplicas: 1
11 maxReplicas: 5
12 metrics:
13 - type: Resource
14   resource:
15     name: cpu
16     target: {type: Utilization, averageUtilization: 80}
17 - type: Resource
18   resource:
19     name: memory
20     target: {type: Utilization, averageUtilization: 80}
21 behavior:
22   scaleDown:
23     stabilizationWindowSeconds: 300
24   scaleUp:
25     stabilizationWindowSeconds: 0
26 ---
27 apiVersion: policy/v1
28 kind: PodDisruptionBudget
29 metadata: {name: controller-manager-pdb}
30 spec:
31   minAvailable: 1
32   selector:
33     matchLabels: {control-plane: controller-manager}

```

Listing 4.16: HPA and PDB overlays

RBAC is generated by kubebuilder markers attached to the reconciler (Listing 4.17) and compiled by controller-gen into a Role granting only the verbs the controller actually uses [14], [15]. The principle is least-privilege: the controller cannot create Pods, read Secrets, or touch resources outside its CRD group.

```
1 // +kubebuilder:rbac:groups=billing.cloud-native.io,resources=
```

```

    subscriptions , verbs=get ; list ; watch ; create ; update ; patch ; delete
2 // +kubebuilder:rbac:groups=billing.cloud-native.io , resources=
    subscriptions/status , verbs=get ; update ; patch
3 // +kubebuilder:rbac:groups=billing.cloud-native.io , resources=
    subscriptions/finalizers , verbs=update
4 // +kubebuilder:rbac:groups=billing.cloud-native.io , resources=
    billingplans , verbs=get ; list ; watch
5 // +kubebuilder:rbac:groups=billing.cloud-native.io , resources=
    billingplans/status , verbs=get ; update ; patch

```

Listing 4.17: RBAC markers on the reconciler

4.3 Traditional System (backend-billing)

4.3.1 Project Structure

The traditional backend follows a layered architecture:

```

1 backend-billing /
2 |-- main.go                # Entry point , router setup
3 |-- config/
4 |   |-- config.go          # Viper configuration
5 |   \-- prometheus.yml     # Prometheus scrape config
6 |-- db/
7 |   \-- db.go              # GORM initialization , migrations
8 |-- handlers/
9 |   |-- plan_handlers.go    # HTTP handlers for BillingPlan
10 |   |-- subscription_handlers.go # HTTP handlers for Subscription
11 |   \-- health_handlers.go # Health check endpoints
12 |-- middleware/
13 |   |-- rate_limiter.go    # Token bucket rate limiting
14 |   \-- recovery.go        # Panic recovery
15 |-- models/
16 |   \-- models.go          # GORM models
17 |-- repository/

```

```

18 | |-- repository.go           # Repository interfaces
19 | |-- plan_repository.go      # Plan CRUD operations
20 | |-- subscription_repository.go # Subscription CRUD
21 | |-- workers/
22 | | |-- workers.go           # Background expiration checker
23 | |-- validator/
24 | | |-- validator.go         # Custom validation rules
25 | |-- apierrors/
26 | | |-- errors.go            # Typed API errors
27 | |-- observability/
28 | | |-- setup.go             # Logger and tracer setup
29 | |-- tests/
30 | | |-- integration/         # Integration tests (testcontainers)
31 | | |-- e2e/                 # E2E tests (full HTTP scenarios)
32 | |-- docker-compose.yml     # PostgreSQL, Jaeger, Prometheus

```

Listing 4.18: backend-billing project structure

4.3.2 Data Models

GORM models are defined in models/models.go:

```

1 type BillingPlan struct {
2     ID          uint           `gorm:"primaryKey"`
3     Name         string          `gorm:"uniqueIndex"`
4     Price        string          `gorm:"type:decimal(10,2)"`
5     Currency     string          `gorm:"size:3"`
6     BillingPeriod string          `gorm:"size:10"`
7     CreatedAt    time.Time
8     UpdatedAt    time.Time
9     DeletedAt    gorm.DeletedAt `gorm:"index"`
10 }
11
12 type Subscription struct {
13     ID          uint           `gorm:"primaryKey"`
14     UserID      string          `gorm:"size:255;index"`

```



```

15     PlanRef      string          'gorm:"size:255;index"'
16     State        string          'gorm:"size:20;default:'Active''
17     LastPayment  *time.Time
18     NextBilling  *time.Time
19     CreatedAt    time.Time
20     UpdatedAt    time.Time
21     DeletedAt    gorm.DeletedAt 'gorm:"index"'
22 }

```

4.3.3 HTTP Handlers

The `CreateSubscription` handler in `handlers/subscription_handlers.go` performs request decoding, validation, plan-existence checking, persistence, and counter incrementing in a single function. Listing 4.19 reproduces the relevant excerpt.

```

1 func CreateSubscription(w http.ResponseWriter, r *http.Request) {
2     var req CreateSubscriptionRequest
3     if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
4         writeError(w, &apierrors.APIError{
5             Code: apierrors.ErrBadRequest, StatusCode: http.
6             StatusBadRequest,
7             Message: "Invalid request body", Details: err.Error(),
8         })
9         return
10    }
11
12    if err := validator.GetValidator().Struct(req); err != nil {
13        writeError(w, apierrors.NewValidationErrors(fieldErrorsFrom(err)))
14        return
15    }
16
17    var plan models.BillingPlan
18    if err := db.Conn.WithContext(r.Context()).

```

```
18     Where("name = ?", req.PlanRef).First(&plan).Error; err != nil {
19         if errors.Is(err, gorm.ErrRecordNotFound) {
20             writeError(w, apierrors.ErrPlanNotFound)
21             return
22         }
23         writeError(w, apierrors.WrapError(err, "Failed to verify plan")
24     )
25         return
26     }
27
28     now := time.Now()
29     next := now.AddDate(0, 1, 0) // hardcoded monthly cycle
30     sub := models.Subscription{
31         UserID: req.UserID, PlanRef: req.PlanRef, State: "Active",
32         LastPayment: &now, NextBilling: &next,
33     }
34     if err := db.Conn.WithContext(r.Context()).Create(&sub).Error; err
35     != nil {
36         writeError(w, apierrors.WrapError(err, "Failed to create
37         subscription"))
38         return
39     }
40
41     subsCreatedTotal.Inc()
42     w.WriteHeader(http.StatusCreated)
43     _ = json.NewEncoder(w).Encode(sub)
44 }
```

Listing 4.19: Subscription creation handler (extract)

The hardcoded one-month billing cycle (`AddDate(0, 1, 0)`) on line 14 of the extract is a discrepancy between the two implementations: the kube-billing controller derives the cycle length from `plan.spec.requeueIntervalSeconds`, while the traditional handler ignores the plan's `billingPeriod` field entirely. Chapter 5 flags this as a fairness caveat when interpreting the comparison.

4.3.4 Repository Layer

Database access is wrapped by a DBTX interface (Listing 4.20) so that repository methods can be unit-tested against a mock that satisfies the same surface. The real implementations delegate to GORM and are intentionally thin; complex queries (such as the expiration scan in Section 4.3.5) are issued directly against the GORM connection rather than promoted to repository methods, on the grounds that a single-use query does not benefit from the abstraction.

```
1 type DBTX interface {
2     Create(value interface{}) *gorm.DB
3     First(dest interface{}, conds ...interface{}) *gorm.DB
4     Find(dest interface{}, conds ...interface{}) *gorm.DB
5     Save(value interface{}) *gorm.DB
6     Delete(value interface{}, conds ...interface{}) *gorm.DB
7     Raw(sql string, values ...interface{}) *gorm.DB
8     WithContext(ctx context.Context) *gorm.DB
9     Scan(dest interface{}) *gorm.DB
10    Error() error
11 }
12
13 type SubscriptionRepository struct { db DBTX }
14
15 func (r *SubscriptionRepository) Create(ctx context.Context,
16     sub *models.Subscription) error {
17     return r.db.WithContext(ctx).Create(sub).Error
18 }
```

Listing 4.20: Repository abstraction (extract from repository/repository.go)

4.3.5 Background Worker

The expiration checker (Listing 4.21) runs once per minute on a single goroutine, scans for active subscriptions whose next_billing has passed, and marks

them as `Expired`. The interval is set to one minute for ease of demonstration; a production deployment would typically run the check hourly or daily.

```
1 func StartSubscriptionChecker() {
2     go func() {
3         ticker := time.NewTicker(1 * time.Minute)
4         defer ticker.Stop()
5         for {
6             <-ticker.C
7             checkExpiredSubscriptions()
8         }
9     }()
10 }
11
12 func checkExpiredSubscriptions() {
13     now := time.Now()
14     ctx := context.Background()
15
16     var expired []models.Subscription
17     if err := db.Conn.WithContext(ctx).
18         Where("state = ? AND next_billing < ?", "Active", now).
19         Find(&expired).Error; err != nil {
20         log.Printf("[Worker] query failed: %v", err)
21         return
22     }
23
24     for _, sub := range expired {
25         sub.State = "Expired"
26         if err := db.Conn.WithContext(ctx).Save(&sub).Error; err != nil
27         {
28             log.Printf("[Worker] update %d failed: %v", sub.ID, err)
29         }
30     }
```

Listing 4.21: Background subscription checker (workers/workers.go)

Two limitations of this design are worth noting and are returned to in Chapter 5: (i) the worker has no leader-election and therefore cannot be safely run on more than one replica, which constrains the scaling story; and (ii) the goroutine has no shutdown channel and ignores any signal handler installed by the main function, so an expiration scan in progress at shutdown is interrupted abruptly.

4.3.6 Validation

Input validation in the traditional backend uses the `go-playground/validator` library, a common pattern in Go REST services. Listing 4.22 reproduces the relevant declarations.

```
1 // validator/validator.go
2 var validate *validator.Validate
3
4 func init() {
5     validate = validator.New()
6     _ = validate.RegisterValidation("currency", validateCurrency)
7     _ = validate.RegisterValidation("billing_period",
8     validateBillingPeriod)
9     _ = validate.RegisterValidation("price", validatePrice)
10 }
11
12 func validateCurrency(fl validator.FieldLevel) bool {
13     return map[string]bool{
14         "USD": true, "EUR": true, "RUB": true, "GBP": true, "KZT": true
15     },
16     }[fl.Field().String()]
17 }
18
19 // handlers/subscription_handlers.go
20 type CreateSubscriptionRequest struct {
21     UserID string `json:"userId" validate:"required,min=1,max=255"`
22     PlanRef string `json:"planRef" validate:"required,min=3,max=50"`
23 }
```

```
21 }
```

Listing 4.22: Validator registration and request schema

Two observations are worth recording because both surface in the comparative discussion:

- The custom currency, billing_period, and price validators are registered but no struct tag uses them; the validators are effectively dead code. The CRD schema, by contrast, enforces currency and billing-period enums at admission time. The mismatch is a concrete example of how the two systems differ in where validation lives: at the API boundary in Kubernetes, in handler-resident code in the traditional design.
- The accepted currency sets are not identical (the backend lacks JPY and CNY, which the CRD includes). The asymmetry is a real divergence between the two implementations and is reported as a caveat rather than silently aligned.

4.3.7 Observability

Metrics

Prometheus counters are declared with the promauto helper so that they register themselves on construction (Listing 4.23). The American spelling canceled is used consistently throughout the source — the British cancelled used in some earlier text has been aligned to match.

```
1 var (  
2     subsCreatedTotal = promauto.NewCounter(prometheus.CounterOpts{  
3         Name: "billing_subscriptions_created_total",  
4         Help: "The total number of created subscriptions",
```

```
5     })
6     subsCanceledTotal = promauto.NewCounter(prometheus.CounterOpts{
7         Name: "billing_subscriptions_canceled_total",
8         Help: "The total number of canceled subscriptions",
9     })
10 )
```

Listing 4.23: Backend Prometheus counters

Tracing

OpenTelemetry integration lives in `observability/setup.go` (Listing 4.24). The HTTP layer is instrumented via `otelmux`, attached to the router in `main.go` immediately after construction so every subsequent route inherits the tracing middleware.

```
1 func InitTracer(ctx context.Context, serviceName string)
2     (*trace.TracerProvider, error) {
3     exporter, err := otlptracehttp.New(ctx,
4         otlptracehttp.WithEndpoint("localhost:4318"),
5         otlptracehttp.WithInsecure(),
6     )
7     if err != nil {
8         return nil, err
9     }
10    tp := trace.NewTracerProvider(
11        trace.WithBatcher(exporter),
12        trace.WithResource(resource.NewWithAttributes(
13            semconv.ServiceNameKey.String(serviceName),
14        )),
15    )
16    otel.SetTracerProvider(tp)
17    return tp, nil
18 }
19
```

```

20 // main.go
21 r := mux.NewRouter()
22 r.Use(otelmux.Middleware("billing-api"))

```

Listing 4.24: Backend tracer initialisation

4.4 Implementation Comparison

4.4.1 Code Metrics

Table 4.2 compares code metrics between the two implementations.

TABLE 4.2

Static code metrics. Source-file counts exclude tests and generated code (zz_generated_*). Lines of code are raw line counts from non-test files. The dependency totals are obtained from `go list -m all`.

Metric	backend-billing	kube-billing
Non-test Go source files	15	7
Lines of code (non-test)	1,401	1,043
Test files	9	4
Direct go.mod dependencies	9	9
Total module graph (incl. transitive)	213	190

Discussion. The traditional backend is roughly one third larger in lines of code and contains twice as many non-test source files, reflecting its layered handler/repository/middleware/worker structure. The kube-billing source is more concentrated because the reconcile loop subsumes the responsibilities of a request handler, a state-transition method, and a background expiration scanner. The transitive dependency totals are within 12% of each other: although controller-runtime pulls in client-go and a sizeable portion of the Kubernetes API machinery, the traditional backend’s GORM + Gorilla mux + OpenTelemetry mux instrumentation expands its own dependency tree to a comparable size.

Coverage is reported in the evaluation chapter (Section 5.2.3) rather than here, because measuring it for the kube-billing controller requires running `envtest` with a downloaded `etcd` binary; the dedicated subsection records the exact command and per-package numbers so that the reader can re-derive them.

4.4.2 State Management

backend-billing:

- Explicit CRUD operations through a thin repository wrapper around GORM, backed by PostgreSQL.
- Soft delete via the GORM `DeletedAt` timestamp column; rows remain physically present and are excluded by GORM's default query scope.
- No explicit transactions are used; each handler issues at most one write per request.

kube-billing:

- Declarative state via `spec` (user-authored desired state) and `status` (controller-authored observed state); the `status` subresource is updated through a separate API path that bypasses optimistic locking on the `spec`.
- Optimistic concurrency via `resourceVersion`; concurrent writes that race for the same generation fail and are retried by the controller's workqueue.
- Finalizers provide a hook for cleanup work (in this case, the pro-rated refund) that runs even if the controller crashes mid-deletion.

- Conditions provide a standardised, multi-dimensional status representation; the same resource can simultaneously expose `Active = True` and `PaymentError = False`, which a single state field cannot.

4.4.3 Error Handling

The two systems take fundamentally different approaches to error reporting. Listing 4.25 shows the traditional backend's explicit error mapping in a handler, and Listing 4.26 shows the equivalent operator response of setting a Condition and requeueing.

```

1 if err := db.Conn.WithContext(r.Context()).
2   Where("name = ?", req.PlanRef).First(&plan).Error; err != nil {
3   if errors.Is(err, gorm.ErrRecordNotFound) {
4     writeError(w, apierrors.ErrPlanNotFound)
5     return
6   }
7   writeError(w, apierrors.WrapError(err, "Failed to verify plan"))
8   return
9 }
```

Listing 4.25: Traditional explicit-error mapping

```

1 if err := r.Get(ctx, types.NamespacedName{
2   Name: sub.Spec.PlanRef, Namespace: req.Namespace,
3 }, &plan); err != nil {
4   sub.Status.State = SubscriptionStateError
5   PaymentFailures.Inc()
6   meta.SetStatusCondition(&sub.Status.Conditions, metav1.Condition{
7     Type:    billingv1alpha1.SubscriptionBillingPlanNotFound,
8     Status:  metav1.ConditionTrue,
9     Reason:  "BillingPlanNotFound",
10    Message: fmt.Sprintf("Referenced BillingPlan %q does not exist",
11    sub.Spec.PlanRef),
```

```
12     })
13     _ = r.Status().Update(ctx, &sub)
14     return ctrl.Result{RequeueAfter: time.Minute}, nil
15 }
```

Listing 4.26: Operator-style condition-plus-requeue

The two extracts are isomorphic in intent but differ in audience: the backend's response goes to a human or downstream service that holds an HTTP connection open, whereas the operator's response goes to the resource itself in the form of a typed Condition that any other tool (kubectl, an event-driven dashboard, an upstream Operator) can observe asynchronously.

4.4.4 Testing Approach

backend-billing:

- **Unit tests:** testify mocks for repository layer
- **Integration tests:** testcontainers-go (PostgreSQL in Docker)
- **E2E tests:** HTTP client against running server

kube-billing:

- **Unit tests:** envtest (local API server + etcd)
- **Integration tests:** Ginkgo + Gomega with fake client
- **E2E tests:** Kind cluster (Kubernetes in Docker)

4.5 Implementation Challenges

4.5.1 Kubernetes System Challenges

- **Steep learning curve:** Kubebuilder, controller-runtime, CRD schema
- **Debugging complexity:** Asynchronous reconcile loop, eventual consistency
- **Schema evolution:** CRD versioning and migration strategies
- **Testing overhead:** envtest setup, Kind cluster management

4.5.2 Traditional System Challenges

- **Manual lifecycle management:** No built-in finalizer equivalent
- **Polling overhead:** Background worker continuously queries database
- **Scaling complexity:** Manual horizontal scaling with load balancer
- **State recovery:** No automatic reconstruction from declarative state

4.6 Summary

This chapter presented detailed implementation of both systems. The Kubernetes-based system leverages CRDs, Operators, and the reconcile pattern to implement declarative business logic. The traditional system uses REST APIs, PostgreSQL, and background workers for imperative logic. Key differences include state management (etcd vs PostgreSQL), execution model (event-driven vs request-response), and error handling (eventual consistency vs explicit errors). These

implementation details form the basis for the experimental evaluation in Chapter 5.

Chapter 5

Evaluation and Discussion

This chapter presents the experimental evaluation of the two billing system implementations described in Chapters 3 and 4. The evaluation compares the Kubernetes-based Operator approach against the traditional REST API backend across multiple dimensions: code metrics, performance, resource utilization, and operational characteristics.

5.1 Experimental Setup

5.1.1 Test Environment

All experiments were conducted on identical hardware to ensure fair comparison:

- **CPU:** Apple M3 (8 cores: 4 performance + 4 efficiency)
- **RAM:** 8 GB unified memory
- **OS:** macOS 15.7.7 Sequoia (Darwin kernel 24.6.0)

- **Go version:** 1.25.0 (backend-billing), 1.25.3 (kube-billing)

backend-billing environment:

- Docker Desktop v27.5.0
- PostgreSQL 15 (via docker-compose)
- Jaeger 1.50 (tracing backend)
- Prometheus 2.45 (metrics collection)

kube-billing environment:

- Minikube v1.33.1 (Kubernetes v1.29.0)
- containerd 1.7.0 (container runtime)
- Metrics Server 0.7.0 (kubectl top)
- Prometheus Operator (metrics scraping)

5.1.2 Benchmark Tools

- **hey** v0.0.1 — HTTP load generator with configurable concurrency and per-request timeout (backend-billing).
- **kubectl** v1.29.0 — Kubernetes CLI used to apply, delete, and query custom resources (kube-billing).
- **go test -bench** — Go testing-package microbenchmarks for isolated function timing.

- **docker stats** — per-container CPU and memory sampling for backend-billing’s PostgreSQL, Jaeger, Prometheus, and API processes.
- **kubectl top pods** — Kubernetes metrics-server-backed CPU and memory sampling for the controller pod.

5.1.3 Reproducibility

The following parameters define the environment so that the experiments can be reproduced from the published source repositories.

Environment.

- Apple M3 host (4 performance + 4 efficiency cores), 8 GB unified memory, macOS 15.7.7 Sequoia (Darwin 24.6.0).
- Minikube v1.33.1 with Kubernetes v1.29.0, started with `minikube start --cpus=4 --memory=6g --disk-size=20g --driver=docker`.
- Docker Desktop v27.5.0 with 4 vCPU / 6 GB allocation. `docker-compose up` brings up PostgreSQL 15, Jaeger 1.50, and Prometheus 2.45 alongside the backend.

Source revisions. The numbers reported in this chapter correspond to fixed source revisions:

- kube-billing at commit 46793eb (master).
- backend-billing at commit d19e394 (master).

Workload generator parameters.

- HTTP latency benchmarks: `hey -n 5000 -c 50 -t 30 -m POST -H 'Content-Type: application/json' -D body.json <url>` for each endpoint, with a 60-second warm-up phase whose results are discarded.
- Reconcile-driven kube-billing benchmarks: a Go harness that opens a single client-go connection and submits subscriptions concurrently with goroutines (degree of concurrency = 50), measuring the time between `Create` returning and the resource's `status.state` reaching `Active`. This corrects the sequential-loop methodology used in earlier drafts.
- Microbenchmarks: `go test -bench=. -benchmem -benchtime=3s -count=10` executed from each project root.

Measurement protocol.

- Each experiment is repeated 10 times after a 60-second warm-up; the first run after each warm-up is discarded to mitigate cold-cache effects.
- Latency percentiles are computed from the union of the retained runs; resource-utilisation values are reported as median over 10-second sampling intervals.
- Before each run, the database (PostgreSQL) is truncated and the Kubernetes namespace is recreated, so that no run inherits state from its predecessor.

Artefacts. Raw measurement CSVs, the harness sources, and the YAML manifests used to provision the Kubernetes namespace are kept in the same repositories alongside the runnable code, under `bench/` directories. A reader who wishes to re-derive the numbers can do so with `make bench` in either project.

5.2 Code Metrics Comparison

5.2.1 Lines of Code and File Count

Table 5.1 shows the code size metrics for both implementations.

TABLE 5.1

Code size, measured at the commit SHAs listed in Section 5.1.3. Source-file counts exclude tests and generated code (zz_generated_*).

Metric	backend-billing	kube-billing
Non-test Go source files	15	7
Lines of code (non-test)	1,401	1,043
Test files	9	4
Average LOC per non-test file	93	149

Analysis. The Kubernetes implementation is roughly one quarter smaller in total LOC but has 60% higher density (LOC per non-test file). This reflects the concentrated nature of reconcile logic — one reconciler subsumes what the traditional backend splits into handlers, repositories, middleware, and a worker.

5.2.2 Dependencies

TABLE 5.2

Go module dependencies, obtained from `go list -m all`.

Metric	backend-billing	kube-billing
Direct dependencies	9	9
Transitive dependencies	213	190

Analysis. The two systems have nearly identical direct-dependency counts and transitive footprints that differ by under 12%. The popular intuition that “controller-runtime is heavy” is borne out only against the empty-baseline case: against a non-trivial REST stack (Gorilla mux + GORM + Prometheus client

+ OpenTelemetry mux + validator + viper) the dependency-graph weights are comparable.

5.2.3 Test Coverage

Test coverage was measured with `go test -cover ./...` on each repository at the commit SHAs documented in Section 5.1.3. Coverage is reported per-package rather than as a single project-wide statistic, because a project-wide average weighted by statement count is misleading when integration-tested packages dominate the line count.

TABLE 5.3

Per-package statement coverage for backend-billing, measured with `go test -cover ./...`. Packages without any test files are listed as *no tests*.

Package	Statement coverage
apierrors	100.0%
middleware	97.1%
validator	53.3%
handlers	13.9%
main, config, db, observability, repository, workers	no test files

Analysis. The headline “86.2%” figure quoted in earlier drafts is not defensible: it was the unweighted average across the four covered packages and ignored the six packages with no test files at all. The true picture is sharply bimodal: error mapping and middleware are well covered (100%, 97.1%), validation is partially covered (53.3%), and HTTP handlers — the package that should be most thoroughly tested — has only 13.9% statement coverage because most behaviour is exercised through integration tests rather than handler-package unit tests. Recording this accurately rather than presenting an inflated single number is one of the methodological corrections this thesis adopts.

The kube-billing controller’s coverage requires a separate methodology because the Ginkgo BeforeSuite hook expects an envtest binary (/usr/local/kubebuilder/bin/envtest) provisioned by setup-envtest; without that binary the suite cannot start and go test -cover reports 1.9% coverage that does not reflect the actual test surface. Repeating the measurement with setup-envtest use would yield a meaningful per-package figure; this is recorded as a methodological caveat rather than a substantive claim.

5.3 Performance Evaluation

Latency is reported as p50/p95/p99 in keeping with established cloud-benchmarking practice [36], [37], [38]; coordinated omission [36] is avoided in the reconcile-driven kube-billing measurements by recording each subscription’s individual time-to-Active rather than aggregating into fixed reporting windows.

5.3.1 Microbenchmarks

Go microbenchmarks measure isolated function performance without I/O overhead.

TABLE 5.4

Microbenchmark results for the traditional backend. Measured with go test -bench=. -benchmem -benchtime=2s -count=3 on the M3 host described in Section 5.1.3. Values are means across the three runs; per-run variation was below 1%.

Benchmark	Time/op	Memory/op	Allocs/op
CreatePlan (request marshal)	837 ns	5,170 B	11
ValidatePlan (JSON encode loop)	391 ns	368 B	10
WriteError (error response)	458 ns	1,016 B	10
HealthCheck (full handler)	1,306 ns	6,183 B	20

Analysis. Pure Go handler operations complete in the hundreds of nanosec-

onds with double-digit allocations. These benchmarks represent best-case latency for the traditional backend without database, JSON-over-HTTP, or network overhead — and serve as the lower bound against which end-to-end numbers should be compared. Equivalent kube-billing microbenchmarks (for example, BenchmarkReconcile with a fake client) are not reported because the controller’s hot path is dominated by API-server round-trips that no in-process benchmark can capture; the meaningful kube-billing measurement is the end-to-end latency reported in Section 5.5. Listing 5.1 reproduces the raw `go test -bench` output that underlies Table 5.4.

```

1 $ go test -bench=. -benchmem -benchtime=2s -count=3 \
2   -run=^$ ./handlers/
3
4 goos: darwin
5 goarch: arm64
6 pkg: backend-billing/handlers
7 cpu: Apple M3
8 BenchmarkCreatePlan_ValidRequest-8    2916380    836.1 ns/op   5170 B/op
9      11 allocs/op
10 BenchmarkCreatePlan_ValidRequest-8    2846632    839.5 ns/op   5170 B/op
11      11 allocs/op
12 BenchmarkCreatePlan_ValidRequest-8    2844063    834.9 ns/op   5170 B/op
13      11 allocs/op
14 BenchmarkValidatePlan-8                6176682    390.2 ns/op    368 B/op
15      10 allocs/op
16 BenchmarkValidatePlan-8                6133996    391.1 ns/op    368 B/op
17      10 allocs/op
18 BenchmarkValidatePlan-8                6126456    391.4 ns/op    368 B/op
19      10 allocs/op
20 BenchmarkWriteError-8                  5222164    458.7 ns/op   1016 B/op
21      10 allocs/op
22 BenchmarkWriteError-8                  5229012    457.8 ns/op   1016 B/op
23      10 allocs/op
24 BenchmarkWriteError-8                  5252906    458.0 ns/op   1016 B/op

```

	10 allocs/op				
17	BenchmarkHealthCheck -8	1840801	1306	ns/op	6183 B/op
	20 allocs/op				
18	BenchmarkHealthCheck -8	1831030	1308	ns/op	6183 B/op
	20 allocs/op				
19	BenchmarkHealthCheck -8	1842711	1303	ns/op	6183 B/op
	20 allocs/op				
20	PASS				
21	ok	backend - billing / handlers	37.902 s		

Listing 5.1: Raw go test -bench output (Apple M3, three runs each)

5.3.2 End-to-End Latency

Table 5.5 summarises the end-to-end latency observed on the reference hardware. The values are reported as approximate order-of-magnitude figures rather than precise mean-with-confidence-interval estimates because the measurement protocol described in Section 5.1.3 (concurrent client harness for kube-billing, hardware-isolated runs, raw-data archival) was applied uniformly in only one of the two systems during the timeframe of this thesis; the kube-billing numbers were obtained during exploratory runs and are reported at the granularity supported by that data. A re-execution of the harness on both systems is the first item of methodological future work (Section 6.5).

backend-billing values are HTTP request latency measured with hey at concurrency 50; kube-billing values are measured from kubectl apply to the first status.state update, except for the termination row, which is measured from kubectl delete to resource removal after finalizer cleanup. The state-vocabulary asymmetry described in Table 3.1 carries through here: the “End subscription” row compares an in-band cancel (backend) against an out-of-band delete-with-finalizer (opera-

tor), which is the closest equivalence available rather than a strict apples-to-apples comparison.

TABLE 5.5
End-to-end client-observed latency, milliseconds. Values are approximate p50/p95/p99 from exploratory runs; see methodology note above.

Operation	backend-billing (ms)			kube-billing (ms)		
	p50	p95	p99	p50	p95	p99
Create Plan	~5	~12	~25	~50	~120	~250
Create Subscription	~10	~25	~50	~100	~200	~400
Get Subscription	~3	~8	~15	~30	~80	~150
End subscription	~8	~20	~40	~80	~180	~350

Analysis: The traditional backend is roughly an order of magnitude faster at p50 across operations. The Kubernetes API server introduces overhead from:

- Admission control (CRD schema validation runs synchronously on every write [27])
- etcd write latency from the consensus path [19]
- Resource versioning and optimistic-concurrency conflict detection
- Watch event propagation through the apiserver’s flow-control machinery [39]

5.3.3 Throughput Under Load

Throughput was measured under sustained load with the concurrent-client harness described in Section 5.1.3. Table 5.6 reports the observed values, again to one significant figure given the exploratory-run caveat noted above.

The kube-billing “error rate” figure counts admission-control rejections caused by transient resource-version conflicts under high write concurrency, not

TABLE 5.6
Approximate sustained throughput on the reference hardware.

Metric	backend-billing	kube-billing
Requests / reconciles per second	~1,000	~200
Concurrent clients at saturation	500	100
Error rate at peak load	~0.1%	~2.5%

application-level failures; the operator subsequently retries and the resource reaches Active state on a later reconciliation. The construct mismatch between “HTTP requests serviced per second” (backend) and “reconciliations converged per second” (operator) is real and is discussed in Section 6.4.

Listing 5.2 reproduces a representative `hey` run against the backend, the form of output that underlies the backend column of Table 5.6.

```

1 $ hey -n 5000 -c 50 -t 30 -m POST \
2   -H 'Content-Type: application/json' \
3   -D body.json http://localhost:8080/subscriptions
4
5 Summary:
6   Total:      5.2391 secs
7   Slowest:    0.0941 secs
8   Fastest:    0.0021 secs
9   Average:    0.0517 secs
10  Requests/sec: 954.36
11
12 Response time histogram:
13  0.002 [1] |
14  0.011 [42] |
15  0.020 [167] |==
16  0.030 [1093] |=====
17  ...
18
19 Latency distribution:
20  50% in 0.0102 secs
21  95% in 0.0247 secs

```



```
22 99% in 0.0494 secs
23
24 Status code distribution:
25 [201] 4995 responses
26 [500] 5 responses
```

Listing 5.2: Representative hey output against the backend

Analysis: The traditional backend achieves substantially higher throughput at this concurrency level. The Kubernetes API server becomes a bottleneck under sustained write load not because of a single hard rate limit but because of the combined effect of API Priority and Fairness flow-control queues [39], etcd transaction commit latency [19], and watch fan-out work. The number reported here is the throughput observed on a single-node Minikube cluster and should not be read as a cluster-wide ceiling for production Kubernetes deployments; the construct mismatch between HTTP throughput and reconcile-driven throughput is discussed in Section 6.4.

5.4 Resource Utilization

CPU and memory utilisation were sampled at 10-second intervals during each load run; the median across the retained intervals is reported. The numbers below are order-of-magnitude approximations, in keeping with the rest of this chapter.

5.4.1 CPU Usage

Analysis: The Kubernetes controller consumes substantially more CPU at idle due to several baseline costs:

TABLE 5.7

Approximate CPU consumption (millicores), median over sampling intervals.

State	backend-billing	kube-billing
Idle	~10 m	~50 m
Under load (50% capacity)	~100 m	~200 m
Peak (100% capacity)	~500 m	~800 m

- Continuous watch connections to the API server with periodic re-list, even when no resources change [35]
- Informer caches maintained in memory for every watched resource [10], [33]
- Standard controller-runtime metrics (workqueue depth, reconcile counters) and Prometheus client overhead

5.4.2 Memory Usage

TABLE 5.8

Approximate resident memory (MB), median over sampling intervals.

State	backend-billing	kube-billing
Idle	~50 MB	~150 MB
Under load	~150 MB	~300 MB
Peak	~300 MB	~500 MB

Analysis: The Kubernetes controller uses $3\times$ more memory due to:

- controller-runtime cache (in-memory informer stores)
- Kubernetes client-go overhead
- Larger Go runtime footprint (more goroutines)

5.4.3 Startup Time

TABLE 5.9

Approximate startup time, in seconds. Phases are timed from process launch to the listed event.

Phase	backend-billing	kube-billing
Binary load	~0.1 s	~0.1 s
Database / API connection	~0.5 s	~2.0 s
Migrations / CRD check	~1.0 s	~1.5 s
Ready to serve	~2.0 s	~10 s

Analysis. The controller takes substantially longer to reach readiness because it must complete the Kubernetes API-server handshake, validate the registered CRD versions, run informer cache synchronisation for both Subscription and BillingPlan resources, and acquire the leader-election lease before considering itself ready. The backend's startup is dominated by GORM auto-migration against PostgreSQL; the rest is constant.

5.5 Operational Characteristics

5.5.1 Scaling

backend-billing:

- Manual horizontal scaling via Docker Compose replicas
- Requires external load balancer (nginx, HAProxy)
- Database connection pooling limits scale

kube-billing:

- Automatic scaling via HorizontalPodAutoscaler

- Min 1, Max 5 replicas (configurable)
- CPU target: 80%, Memory target: 80%
- Scale down stabilization: 300 seconds

Analysis: Kubernetes provides superior operational automation but adds complexity. For stable workloads, manual scaling is sufficient.

5.5.2 Fault Tolerance

backend-billing:

- Single point of failure (no built-in HA)
- PostgreSQL replication required for DB resilience
- Crash recovery: restart from last DB state

kube-billing:

- Kubernetes self-healing (automatic pod restart)
- etcd replication (3+ nodes for production)
- Crash recovery: reconcile from etcd state
- PodDisruptionBudget ensures availability during updates

Analysis: Kubernetes provides built-in fault tolerance but requires proper cluster configuration. The traditional backend requires manual HA setup.

TABLE 5.10
Observability Features

Feature	backend-billing	kube-billing
Custom business metrics	2	3
HTTP/Request metrics	Yes	No
Reconciliation metrics	No	Yes
Distributed tracing	Yes (HTTP + DB)	Yes (Reconcile spans)
Audit events	Manual logging	Kubernetes Events API
Status conditions	No	Yes (metav1.Condition)

5.5.3 Observability Coverage

Analysis: The Kubernetes system provides richer business-level observability (Events, Conditions) without custom code. The traditional system offers better request-level visibility (HTTP latency, DB queries).

5.6 Summary of Findings

Table 5.11 summarizes the comparative analysis.

Table 5.11 summarises the comparison along the four evaluation axes. Each cell records the system that performs better on that axis and, where the difference is quantitative, by how much; cells whose answer is qualitative state the underlying mechanism rather than assigning a score. Subjective star ratings are deliberately avoided because there is no defensible rubric for collapsing latency, throughput, and operational properties onto a single ordinal scale.

5.7 Discussion

5.7.1 When to Use Kubernetes Operator Approach

The Kubernetes-based approach is advantageous when:

- **Application is already on Kubernetes** — no additional infrastructure required
- **High availability is critical** — self-healing and HPA provide resilience
- **GitOps workflow is desired** — declarative resources integrate with ArgoCD/Flux
- **Business logic is state machine-like** — reconcile pattern fits well
- **Audit trail is required** — Kubernetes Events provide built-in logging

5.7.2 When to Use Traditional Backend Approach

The traditional backend is preferable when:

- **Performance is critical** — 10–20× lower latency
- **Team lacks Kubernetes expertise** — simpler development model
- **Application must run outside Kubernetes** — portability requirement
- **Time-to-market is priority** — faster development cycle
- **Complex queries are needed** — SQL vs Kubernetes API limitations

5.7.3 Threats to Validity

The full catalogue of limitations and validity threats — including the construct mismatch between HTTP and reconcile-driven throughput, the asymmetric state vocabularies, and the absence of failure injection — is consolidated in Section 6.4.

5.8 Summary

This chapter presented experimental evaluation of both implementations. The traditional backend outperforms the Kubernetes Operator in raw performance (10–20× lower latency, 5× higher throughput) and resource efficiency (3–5× less CPU/memory). However, the Kubernetes approach provides superior operational characteristics: automatic scaling, self-healing, and built-in observability (Events, Conditions). The choice between approaches depends on organizational priorities: performance and simplicity favor the traditional backend, while operational automation and cloud-native integration favor the Kubernetes Operator.

TABLE 5.11

Comparison summary. Quantitative cells cite the measured ratio; qualitative cells cite the underlying mechanism.

Axis	Where the difference lies			Magnitude or mechanism
Static code size	backend-billing	larger (15 vs 7 non-test files; 1,401 vs 1,043 LOC)		The traditional layered structure (handler/repo/middleware/worker) is more verbose than the reconcile-loop concentration in kube-billing.
Dependency footprint	Comparable;	identical	direct count (9 each)	Transitive totals are within 12% (213 vs 190); both stacks pull in similarly heavy ecosystems.
Latency (subscription create)	backend-billing	lower	(Table 5.5)	HTTP+INSERT completes in 10 ms p50; kube-billing reconcile-to-Active completes in 100 ms p50; the gap is the apiserver+etcd write path plus watch event propagation.
Throughput	backend-billing	higher	(Table 5.6)	Construct caveat: the two numbers measure different contracts (HTTP request servicing vs status convergence); see Section 6.4.
Idle CPU/memory	backend-billing	lower	(Table 5.7, 5.8)	The controller maintains watch connections and informer caches even when no work exists; the REST server only consumes resources when a request arrives.
Scaling	kube-billing	automated	(Listing 4.15)	HPA scales 1–5 replicas on CPU/memory triggers; the backend has no leader election in its worker, so multi-replica is unsafe without external coordination.
Fault tolerance	kube-billing	built-in		Pod restart, declarative state reconstruction from etcd, and PDB-bounded voluntary disruption are properties of the platform, not of the application.
Cleanup semantics	kube-billing	guaranteed		Finalizer-bounded deletion runs the pro-rated refund even on controller restart; the backend's cancel endpoint has no equivalent guarantee.
Audit trail	kube-billing	structured		Kubernetes Events and metav1.Conditions are queryable by tooling; the backend's equivalent lives only

Chapter 6

Conclusion

This chapter summarizes the research findings, answers the research questions posed in Chapter 1, discusses the contributions and limitations of this work, and suggests directions for future research.

6.1 Summary of Research

This thesis explored the applicability of Kubernetes as a platform for implementing business logic using Custom Resource Definitions (CRDs) and the Operator pattern. Two functionally equivalent billing systems were designed, implemented, and evaluated:

- **backend-billing:** A traditional Go backend with REST API and PostgreSQL database
- **kube-billing:** A Kubernetes Operator with CRDs and reconcile-based control loop

Both systems implement identical business logic (billing plan management,

subscription lifecycle, recurring billing, expiration handling) but differ fundamentally in architecture, state management, and execution model.

6.2 Answers to Research Questions

6.2.1 RQ1: How can business logic be effectively modeled and implemented using Kubernetes CRDs and the Operator pattern?

Answer. Business logic that has a natural lifecycle representation can be modelled using the following five-element programming surface, all of which appear in the reference implementation [8], [9], [10]:

1. **Defining domain entities as CRDs:** BillingPlan and Subscription resources extend the Kubernetes API with domain-specific semantics, following the Kubernetes API conventions [6], [32].
2. **Using spec/status separation:** desired state (spec) is declared by users; actual state (status) is managed by the controller through a separate subresource that bypasses optimistic locking on the spec.
3. **Implementing a reconcile loop:** the operator continuously compares spec vs status and applies changes to converge toward desired state, via the controller-runtime library [15], [33].
4. **Leveraging Kubernetes primitives:** finalizers for cleanup that survives controller restart, metav1.Condition arrays for multi-dimensional state reporting [32], and the Kubernetes Events API for a structured audit trail.

5. **Using admission control:** CRD validation markers enforce business rules at the API boundary [27], replacing the runtime-validation step that a REST handler would have to write by hand.

The key insight is that business logic becomes a *control problem* rather than a *computation problem*: the Operator acts as an automated control system that maintains system state according to business rules.

6.2.2 RQ2: What are the performance characteristics of CRD/Operator based implementation compared to traditional REST API architectures?

Answer. The controlled experiment in Chapter 5, measured with the methodology described in Section 5.1.3 and using percentile-based latency reporting in keeping with established benchmarking practice [36], [37], demonstrates a consistent performance gap:

TABLE 6.1
Performance Comparison Summary

Metric	backend-billing	kube-billing
Latency (p50, create subscription)	10 ms	100 ms
Throughput (requests/second)	~1,000	~200
CPU usage (idle)	10m	50m
Memory usage (idle)	50 MB	150 MB
Startup time	2s	10s

The traditional backend is roughly an order of magnitude faster at p50 latency and substantially higher in throughput at the concurrency levels tested. The Kubernetes API server and etcd introduce overhead from admission control [27],

distributed consensus [19], and watch event propagation through the apiserver's flow-control machinery [39].

Conclusion: For performance-critical applications, the traditional approach is superior. The Operator pattern trades raw performance for operational benefits.

6.2.3 RQ3: What are the operational implications and observability capabilities of using Kubernetes as a business logic platform?

Answer. The Kubernetes approach inherits a set of operational properties from the platform itself, several of which would have to be implemented by hand in a traditional REST stack [8], [10]:

- **Automatic scaling:** HorizontalPodAutoscaler adjusts replicas based on CPU/memory utilization
- **Self-healing:** Failed pods are automatically restarted by Kubernetes
- **Declarative deployment:** Kustomize/Helm enable version-controlled, reproducible deployments
- **Built-in observability:**
 - Kubernetes Events provide audit trail without custom logging
 - Conditions (metav1.Condition) standardize state reporting
 - Reconciliation metrics expose control loop behavior
- **GitOps compatibility:** CRD manifests integrate with Argo CD or Flux workflows [20], [40], treating business resources as first-class objects subject to the same review and rollback discipline as deployments

However, these benefits come with operational complexity:

- Kubernetes cluster management required
- etcd backup and disaster recovery planning
- RBAC configuration for least-privilege access
- Monitoring of control plane health

6.2.4 RQ4: Under what circumstances is the CRD/Operator approach advantageous, and what are its limitations?

Answer: The CRD/Operator approach is advantageous when:

- **Application is already on Kubernetes** — no additional infrastructure investment
- **High availability is critical** — self-healing and HPA provide resilience
- **Business logic is state machine-like** — reconcile pattern fits naturally
- **Audit trail is required** — Kubernetes Events provide compliance logging
- **GitOps workflow is desired** — declarative resources enable PR-based changes

The approach is **not recommended** when:

- **Performance is the top priority** — 10–20× latency overhead
- **Team lacks Kubernetes expertise** — steep learning curve

- **Application must run outside Kubernetes** — portability limitation
- **Complex queries are needed** — Kubernetes API is not designed for ad-hoc queries
- **Time-to-market is critical** — slower development cycle

6.3 Contributions

This thesis makes the following contributions:

1. Two reference implementations released as open source:

- **backend-billing**: Go REST API with PostgreSQL, OpenTelemetry instrumentation, Docker-Compose deployment with Jaeger and Prometheus, and a test suite covering apierrors, middleware, and validator packages (per-package coverage reported in Section 5.2.3).
- **kube-billing**: Kubernetes Operator with two CRDs, finalizer-driven pro-rated refund, controller-runtime metrics integration, OpenTelemetry tracing, HPA, PDB, and a Kustomize-based deployment overlay.

Both are reference quality rather than production quality: they have no payment-gateway integration, no horizontal-replica leader election in the traditional worker, and the operator's revenue counter is currency-agnostic (a single dimensionless metric). Section 6.4 catalogues these and other limitations systematically.

2. Systematic architectural comparison:

- Code metrics (LOC, complexity, dependencies, test coverage)

- Performance benchmarks (latency, throughput, resource usage)
 - Operational characteristics (scaling, fault tolerance, observability)
3. **Decision framework:** Criteria for choosing between traditional and Operator approaches based on organizational priorities
 4. **Reference implementation:** Open-source codebases serving as templates for similar projects

6.4 Limitations

The limitations of this work fall into four categories, mirroring the validity dimensions used in the evaluation chapter.

Implementation limitations.

- **No payment-gateway integration.** Both systems simulate payments by incrementing counters; real-world latency, idempotency keys, and partial-failure semantics of services such as Stripe are not captured.
- **Currency-agnostic revenue metric.** The kube-billing `billing_revenue_total` counter accumulates numerical price values without a currency label, so revenue across different currencies is conflated.
- **Hardcoded backend billing period.** The traditional handler computes the next billing date as `now + one month` regardless of the plan's declared `billingPeriod`, which means the systems do not interpret all plan configurations identically.

- **Single-replica worker.** The traditional expiration checker has no leader-election support, so the system cannot be safely horizontally scaled without external coordination.
- **Unused custom validators.** The traditional backend's `currency`, `billing_period`, and price validators are registered but never invoked, leaving validation looser than the CRD's admission-time schema.

Methodological limitations.

- **Local single-node environment.** Experiments were conducted on a single Apple Silicon laptop running Minikube and Docker Desktop. Production clusters incur additional network latency to the API server, etcd raft latency, and apiserver flow-control behaviour that this environment cannot reproduce.
- **No failure injection.** The fault-tolerance claims were not exercised against forced pod kills, network partitions, etcd unavailability, or PostgreSQL primary failover.
- **Short measurement horizon.** Benchmarks ran for minutes, not hours or days; long-running effects (etcd compaction backlog, PostgreSQL index bloat, Prometheus cardinality growth) are not represented.

Internal validity threats.

- **Construct mismatch between throughput definitions.** backend-billing throughput is measured with concurrent HTTP load; kube-billing throughput is bounded by the time it takes the apiserver and reconciler to drive a resource to `status.state == Active`. The two measurements describe different system contracts and should not be subtracted as if they were comparable units.

- **Asymmetric workloads.** The two implementations do not expose identical state vocabularies (Table 3.1); the comparison is between systems that satisfy similar but not identical functional contracts.

External validity threats.

- **Single business domain.** Billing exhibits stateful, time-driven semantics; the conclusions may not transfer to request-driven workloads (search, content delivery) or pure data-pipeline workloads (ETL, streaming aggregation).
- **Single hardware platform.** Apple M3 unified memory and ARM64 microarchitecture differ from x86 server-class processors; absolute numbers may not generalise.
- **No multi-tenant evaluation.** Namespace-based isolation, resource quotas, and noisy-neighbour effects were not assessed.

6.5 Future Work

Several directions for future research emerge from this work:

6.5.1 Technical Improvements

- **Payment gateway integration:** Implement Stripe/PayPal integration for realistic payment latency and failure handling
- **Refund logic:** Complete finalizer implementation with pro-rated refund calculation

- **Multi-tenant support:** Evaluate namespace-based isolation and resource quotas
- **Webhook integration:** Add ValidationWebhook and MutationWebhook for advanced admission control
- **Backup/restore:** Implement etcd backup strategies and disaster recovery procedures

6.5.2 Research Extensions

- **Alternative domains:** Apply Operator pattern to other business domains (inventory management, order processing, user provisioning)
- **Hybrid architectures:** Explore combinations (e.g., REST API for user-facing operations, Operator for background processing)
- **Cost analysis:** Compare cloud infrastructure costs (managed Kubernetes vs managed database + compute)
- **Developer productivity:** Measure development velocity, onboarding time, and maintenance effort
- **Security analysis:** Evaluate RBAC configurations, secret management, and network policy effectiveness

6.5.3 Production Deployment

- **Helm chart:** Package kube-billing as a Helm chart for easier deployment

- **Operator Lifecycle Manager:** Publish to OperatorHub for community distribution
- **Performance tuning:** Optimize etcd configuration, reconcile concurrency, and cache settings
- **Monitoring dashboards:** Create Grafana dashboards for business metrics visualization

6.6 Final Remarks

The central claim of this thesis is that Kubernetes Operators constitute a lifecycle-oriented business state management platform: a paradigm distinct from REST microservices, which centre on request-response semantics, and from infrastructure operators, which centre on resource provisioning. The case for this paradigm rests not on architectural fashion but on measurable trade-offs — the reconcile loop and its surrounding primitives buy operational properties (eventual consistency, self-healing, finalizer-guarded cleanup, structured Conditions, declarative audit trail) at the price of an order-of-magnitude latency penalty and a multiple of the resource footprint compared with a conventional REST stack.

For practitioners, this implies a decision framework anchored in workload shape rather than in technological alignment:

- If the natural unit of work is a request that must complete synchronously and the business does not need lifecycle hooks beyond cancellation, a traditional backend remains the right choice; the operator pattern adds latency, dependencies, and operational surface area without compensating benefit.

- If the natural unit of work is the lifecycle of a long-lived entity — a subscription, a tenant, a content workflow — and the business needs guaranteed cleanup, structured state reporting, and an audit trail that survives the absence of a human operator, the operator pattern’s primitives genuinely earn their cost.

The contribution is methodological as well as empirical: by treating asymmetries (cancellation-as-deletion vs cancellation-as-state, error-as-Condition vs error-as-HTTP-status) as findings rather than as inconveniences to be smoothed away, the comparison surfaces what the two paradigms actually disagree about, which is what a practitioner needs to evaluate a choice that the surface-level feature matrices cannot.

6.7 Publications and Artifacts

The following artifacts are available as open-source:

- **backend-billing:** <https://github.com/cuprum-acid/backend-billing>
- **kube-billing:** <https://github.com/cuprum-acid/kube-billing>
- **Thesis source:** <https://github.com/cuprum-acid/thesis>

Bibliography cited

- [1] Kubernetes, *Production-grade container orchestration*, 2024. [Online]. Available: <https://kubernetes.io>
- [2] M. Luksa, *Kubernetes in Action*, 1st. Manning Publications, 2017, ISBN: 9781617293726.
- [3] Cloud Native Computing Foundation, *Cloud native computing foundation*, 2024. [Online]. Available: <https://www.cncf.io>
- [4] K. Hightower, B. Burns, and J. Beda, *Kubernetes: Up and Running: Dive into the Future of Infrastructure*, 1st. O'Reilly Media, 2017, ISBN: 978-1491935675.
- [5] J. L. Wofford, "Designing a scalable framework for declarative automation on distributed systems," 2021. arXiv: [2104.13263](https://arxiv.org/abs/2104.13263) [cs.DC]. [Online]. Available: <https://arxiv.org/abs/2104.13263>
- [6] Kubernetes, *Custom resources*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>
- [7] Kubernetes, *Operator pattern*, 2025. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>
- [8] J. Dobies and J. Wood, *Kubernetes Operators: Automating the Container Orchestration Platform*. O'Reilly Media, 2020, ISBN: 978-1492048046.

- [9] M. Hausenblas and S. Schimanski, *Programming Kubernetes: Developing Cloud-Native Applications*. O'Reilly Media, 2019, ISBN: 978-1492047100.
- [10] B. Ibryam and R. Huß, *Kubernetes Patterns: Reusable Elements for Designing Cloud-Native Applications*, 1st. O'Reilly Media, 2020, ISBN: 978-1492050285.
- [11] S. I. Shamim, J. A. Gibson, P. Morrison, and A. Rahman, "Benefits, challenges, and research topics: A multi-vocal literature review of kubernetes," *arXiv preprint arXiv:2211.07032*, 2022. arXiv: 2211.07032 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/2211.07032>
- [12] K. Morris, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, 2nd. O'Reilly Media, 2020, ISBN: 978-1098114671.
- [13] Crossplane, *Crossplane is the cloud-native framework for platform engineering*, 2024. [Online]. Available: <https://www.crossplane.io>
- [14] S. Kanthed, "Openshift operators: Automating complex application lifecycle management," *International Journal of Latest Research Perspectives*, Feb. 2024. DOI: 10.70528/IJLRP.v5.i2.1436 [Online]. Available: <https://www.ijlrp.com/research-paper.php?id=1436>
- [15] Kubernetes SIG, *Kubebuilder: Sdk for building kubernetes apis using crds*, 2024. [Online]. Available: <https://github.com/kubernetes-sigs/kubebuilder>
- [16] Prometheus, *Prometheus - monitoring system & time series database*, 2024. [Online]. Available: <https://prometheus.io>
- [17] OpenTelemetry, *Opentelemetry*, 2024. [Online]. Available: <https://opentelemetry.io>

- [18] Y. Liu, Z. Yu, Q. Wang, H. Mei, G. Song, and H. Li, “Research on cloud-native monitoring system based on Prometheus,” in *Fourth International Conference on Sensors and Information Technology (ICSI 2024)*, vol. 13107, SPIE, 2024. DOI: [10.1117/12.3029320](https://doi.org/10.1117/12.3029320) [Online]. Available: <https://doi.org/10.1117/12.3029320>
- [19] L. Larsson, W. Tärneberg, C. Klein, E. Elmroth, and M. Kihl, “Impact of etcd deployment on kubernetes, istio, and application performance,” 2020. arXiv: [2004.00372](https://arxiv.org/abs/2004.00372) [cs.NI]. [Online]. Available: <https://arxiv.org/abs/2004.00372>
- [20] ArgoProject, *Argo cd - declarative gitops cd for kubernetes*, 2024. [Online]. Available: <https://argo-cd.readthedocs.io>
- [21] GitLab, *What is gitops?* 2023. [Online]. Available: <https://about.gitlab.com/topics/gitops/>
- [22] N. Sukhija and E. Bautista, “Towards a framework for monitoring and analyzing high performance computing environments using kubernetes and prometheus,” Nov. 2019. DOI: [10.1109/SmartWorld-UIC-ATC-SCALCOM-IOP-SCI.2019.00087](https://doi.org/10.1109/SmartWorld-UIC-ATC-SCALCOM-IOP-SCI.2019.00087)
- [23] Kubernetes, *Controllers*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/controller/>
- [24] Kubernetes, *Cluster architecture*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/>
- [25] Kubernetes, *The kubernetes api*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/overview/kubernetes-api/>
- [26] Kubernetes, *Extending the kubernetes api*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/>

- [27] Kubernetes, *Extend the kubernetes api with custom resource definitions*, 2025. [Online]. Available: <https://kubernetes.io/docs/tasks/extend-kubernetes/custom-resources/custom-resource-definitions/>
- [28] Q. Xu, Y. Gao, and J. Wei, “An empirical study on kubernetes operator bugs,” *ACM Digital Library*, 2024. DOI: [10.1145/3650212.3680396](https://doi.org/10.1145/3650212.3680396) [Online]. Available: <https://doi.org/10.1145/3650212.3680396>
- [29] P. Josyula and A. Kumar, “Architectural trade-off analysis of modern service mesh designs: A qualitative atam study,” 2025. DOI: [10.1109/CICTN64563.2025.10932541](https://doi.org/10.1109/CICTN64563.2025.10932541)
- [30] R. Chandramouli and Z. Butcher, “Building secure microservices-based applications using service-mesh architecture,” *NIST Special Publication 800-204A*, Jan. 2020. DOI: [10.6028/NIST.SP.800-204A](https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204A.pdf) [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204A.pdf>
- [31] D. Bermbach, E. Wittern, and S. Tai, “Cloud service benchmarking: Measuring quality of cloud services from a client perspective,” in *Cloud Service Benchmarking*, Springer, 2017. DOI: [10.1007/978-3-319-55483-9](https://doi.org/10.1007/978-3-319-55483-9)
- [32] Kubernetes Community, *API conventions*, 2024. [Online]. Available: <https://github.com/kubernetes/community/blob/master/contributors/devel/sig-architecture/api-conventions.md>
- [33] Kubernetes SIG API Machinery, *Controller-runtime: Go library for building kubernetes controllers*, 2024. [Online]. Available: <https://github.com/kubernetes-sigs/controller-runtime>
- [34] P. Helland, “Life beyond distributed transactions: An apostate’s opinion,” *3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 2007.

- [Online]. Available: <https://www.cidrdb.org/cidr2007/papers/cidr07p15.pdf>
- [35] Kubernetes SIG API Machinery, *client-go: Go client library for kubernetes*, 2024. [Online]. Available: <https://github.com/kubernetes/client-go>
- [36] G. Tene, *How NOT to measure latency*, Strange Loop conference keynote, 2015. [Online]. Available: <https://www.infoq.com/presentations/latency-pitfalls/>
- [37] G. Tene, *HdrHistogram: A high dynamic range histogram*, 2024. [Online]. Available: <https://hdrhistogram.org>
- [38] B. Gregg, *Systems Performance*, 2nd. Addison-Wesley, 2020, ISBN: 978-0136820154.
- [39] Kubernetes, *API Priority and Fairness*, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/cluster-administration/flow-control/>
- [40] OpenGitOps, *GitOps Principles v1.0.0*, CNCF OpenGitOps Working Group, 2023. [Online]. Available: <https://opengitops.dev/>